

中图分类号:

UDC:

学校代码: 10055

密级: 公开

南 开 大 学
硕 士 学 位 论 文

基于主动学习的微服务调用链辅助标注机制研究

Research on an Active Learning-Based Auxiliary Labeling

Mechanism for Traces in Microservice

论文作者 车泽宇

指导教师 张圣林 副教授

申请学位 工学硕士学位

培养单位 软件学院

学科专业 软件工程

研究方向 智能运维

答辩委员会主席 xxx 教授

评阅人 xxx xxx xxx xxx
xxx 教授

南开大学研究生院

二〇二五年五月

南开大学学位论文使用授权书

本人完全了解《南开大学关于研究生学位论文收藏和利用管理办法》关于南开大学（简称“学校”）研究生学位论文收藏和利用的管理规定，即：

南开大学拥有在著作权法规定范围内学位论文的使用权，其中包括：

已获学位的研究生必须按学校规定提交学位论文，学校可以采用影印、缩印或其他复制手段保存学位论文；学校根据规定向教育部指定的收藏和存档单位提交学位论文；

为教学和科研目的，学校可以将公开的学位论文作为资料在图书馆、资料室等场所供校内师生阅读，或在校园网范围内供师生检索、浏览；

非公开学位论文在解密后的使用权同公开论文。

本人承诺：本人的学位论文是在南开大学学习期间创作完成的作品，并已通过论文答辩；提交的学位论文电子版与纸质本论文的内容一致，如因不同造成不良后果由本人自负。

本人签署本授权书一份（此授权书为论文中一页），交图书馆留存。

学位论文作者暨授权人（亲笔）签字：

2025 年 5 月 25 日

南开大学研究生学位论文作者信息

论 文 题 目	基于主动学习的微服务调用链辅助标注机制研究				
姓 名	车泽宇	学 号	2120230747	答 辩 日 期	20 25 年 5 月 25 日
论 文 类 别	博士 ☐ 学历硕士 ☒ 专业学位硕士 ☐ 同等学力硕士 ☐ 划 ☒ 选择				
学院（单位）	软件学院		学科/专业（专业学位）名称		软件工程
联 系 电 话	18822294226		电 子 邮 箱	chezeyu@mail.nankai.edu.cn	
通讯地址（邮编）：天津滨海新区宏达街泰达学院，300457					
非公开论文编号			备 注		

注：本授权书适用我校授予的所有博士、硕士的学位论文。如已批准为非公开学位论文，须向图书馆提供批准通过的《南开大学研究生申请非公开学位论文审批表》复印件和“非公开学位论文标注说明”页原件。

南开大学学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师指导下进行研究工作所取得的研究成果。除文中已经注明引用的内容外，本学位论文的研究成果不包含任何他人创作的、已公开发表或者没有公开发表的作品的内容。对本论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明。本学位论文原创性声明的法律 responsibility 由本人承担。

学位论文作者签名：_____

2025 年 5 月 25 日

非公开学位论文标注说明

(本页表中填写内容须打印)

根据南开大学有关规定，非公开学位论文须经指导教师同意、作者本人申请和相关部门批准方能标注。未经批准的均为公开学位论文，公开学位论文本说明为空白。

论文题目			
申请密级	☐ 限制 (≤ 2 年) ☐ 秘密 (≤ 10 年) ☐ 机密 (≤ 20 年)		
保密期限	20 年 月 日至 20 年 月 日		
审批表编号		批准日期	20 年 月 日

南开大学学位评定委员会办公室盖章 (有效)

注：限制 ★2 年 (可少于 2 年)；秘密 ★10 年 (可少于 10 年)；机密 ★20 年 (可少于 20 年)

摘要

调用链（Trace）数据在微服务系统架构中起到了很重要的系统可观测以及可靠性维护的作用。在智能运维领域，很多先进的运维算法都严重依赖于调用链数据，包括异常检测和根因分析等任务。然而，由于缺少高质量标注的数据集作为有效的训练和评估，现有算法在实际应用中往往难以达到预期性能。在大规模的微服务系统中，每天都会有数十亿的调用链数据产生，对于人工手动标注数据的开销将会非常大，导致很难获取到高质量的调用链数据集。

针对这一问题，本文创新性提出了 LabelEase，这是一个半自动化的调用链标注工具。LabelEase 使用主动学习技术，针对调用链数据集实现高效且准确的标注。对于调用链异常标注来说，LabelEase 结合基于图的调用链表征技术对相似的调用链数据进行聚类，然后挑选一小部分最有代表性的调用链数据用于人工标注，而不需要标注所有的调用链数据。对于微服务根因标注来说，LabelEase 对标注好的异常调用链数据进行聚合，利用谱分析故障定位技术计算每个微服务节点的异常分数辅助运维工程师进行故障的标注。本文基于两个大规模的数据集进行了系统的实验评估 LabelEase 的表现。实验结果表明 LabelEase 在异常标注任务上均实现了超过 0.98 的 F_1 -score，并在根因标注任务上实现了 0.89 的故障标注精准率。与此同时，LabelEase 减少了超过 99.9% 人工标注的开销成本。据调研所知，本文第一个提出了针对调用链数据集的半自动化辅助标注工具，实现了高效且准确的调用链数据标注。

关键词：数据标注工具；调用链异常检测；根因定位

Abstract

Trace data is crucial for system observability and maintainability within microservices architectures, and many operation algorithms depend heavily on trace data, including anomaly detection, root cause analysis, *etc.* However, the actual performance of these algorithms might be unsatisfactory due to the absence of high-quality labeled datasets for effective training and evaluation. Since billions of traces could be generated daily for large-scale microservices, labeling overhead is the main hurdle to obtaining high-quality trace datasets.

In this paper, we propose LabelEase, a novel semi-automatic trace labeling tool, which uses active learning techniques to achieve efficient and accurate trace labeling. For anomaly trace labeling, LabelEase clusters similar traces with a graph-based trace representation technique and selects a few representative traces for human labeling, avoiding labeling most of the traces. For root cause labeling, LabelEase aggregates the labeled anomalous traces and identifies the service’s failures for operators to label. Our systematic experiments on two large-scale datasets show that LabelEase achieves over 0.98 F_1 -score in anomaly trace labeling and 0.89 precision of failure detection in root cause labeling, LabelEase can reduce operators’ labeling overhead by more than 99.9%. To the best of our knowledge, we are the first to propose a semi-automatic trace labeling tool capable of achieving efficient and accurate trace labeling.

Key Words: Data labeling tool; Trace anomaly detection; Root cause localization

目录

第一章 绪论	1
第一节 研究背景与意义	1
第二节 国内外研究现状	2
一、分布式追踪技术	2
二、KPI 异常标注技术研究现状	3
三、基于调用链的异常检测研究现状	4
四、基于调用链的根因定位研究现状	5
五、高质量数据集的启示：以 ImageNet 为例	6
第三节 研究内容及主要贡献	7
第四节 论文组织结构	9
第二章 相关基础知识	11
第一节 微服务系统架构概述	11
一、核心技术特征	11
二、服务间通信与交互机制	12
第二节 调用链	13
第三节 异常检测与根因分析：微服务运维的双重范式	14
一、异常检测	14
二、根因分析	15
第四节 异常标注中的相关基础知识	16
一、主动学习	16
二、图神经网络与表征学习基础	16
三、聚类	18
第五节 Django 后端开发框架	19
一、MVT 架构模式	19
二、在 AIOps 科研工具中的适用性	20

第六节 Vue.js 前端开发框架	20
一、MVVM 模式与响应式原理	20
二、在调用链标注任务中的优势	21
第七节 本章小结	21
第三章 基于图表征与主动学习的异常标注	22
第一节 引言	22
第二节 方法介绍	23
一、整体流程概述	23
二、基于图的调用链表征	24
三、异常标注模块	27
第三节 Trace 异常标注工具使用和介绍	28
第四节 数据和对比方法	29
一、数据集	29
二、对比方法介绍	30
三、实验环境与超参数	30
四、评价指标	31
第五节 实验结果及分析	32
一、LabelEase 与基准算法的对比分析 (RQ1)	32
二、基于图的调用链表征效果分析 (RQ2)	34
三、聚类策略的有效性分析 (RQ3)	35
四、待标注样本数量的敏感性分析 (RQ4)	36
第六节 本章小结	37
第四章 基于 Trace 的微服务系统根因标注	39
第一节 引言	39
第二节 方法介绍	41
一、整体流程概述	41
二、异常数据聚合与时间窗口划分	41
三、基于谱系故障定位的微服务异常分数计算	42

四、多维特征融合与专家辅助决策.....	44
第三节 微服务系统根因标注工具使用和介绍	44
第四节 实验结果及分析	47
第五节 本章小节.....	48
第五章 总结与展望	50
第一节 工作总结.....	50
第二节 未来展望.....	51
参考文献.....	53
致谢	59
个人简历、在学期间发表的学术论文及研究成果.....	60

第一章 绪论

本章首先描述了基于调用链数据对多微服务系统进行运行维护的背景和重要性；之后概述了当前在智能运维领域基于调用链分析和数据标注的国内外研究现状；然后总结了本文研究的主要内容与主要贡献；最后叙述了论文的组织结构。

第一节 研究背景与意义

随着互联网科学技术的发展，如今微服务系统架构已经成为构建现代软件系统的主流方案^[1-3]。微服务系统将原来庞大的单体应用拆分成一组松耦合、能够独立部署的微服务^[4-6]。大规模的微服务系统能够包含成数万个微服务^[7]。面对如此复杂且庞大的系统架构，微服务系统产生的调用链（Trace）数据对于理解系统表现，诊断性能问题以及确保可靠性和稳定性是非常重要的^[8]。通过分析调用链数据，运维工程师可以看清服务之间的调用关系，快速发现性能瓶颈和异常行为，从而在动态变化的运行环境中持续优化系统的整体表现^[9-11]。

在实际运维中，要想通过调用链数据做好异常检测和根因分析，学术界其实已经提出了不少算法^[6,12-15]。但真要用到生产环境里，选出一个合适的算法并不容易，主要卡在几个现实问题上。一方面，现在公开能用的调用链数据集，像 Train Ticket^[16]、NetManAIOps^[17]这些，规模都不大，覆盖的故障场景也比较单一，用于评估算法的效果往往不够全面。另一方面，不同业务场景下的调用链结构差异很大，算法的效果如何，越要用实际的业务数据进行评估。可是，每个系统都有自己的特点，通用的数据集很难直接反映真实的情况。虽然现在的异常检测和根因分析算法大多是无监督或半监督的技术路线，不需要太多标注数据，但如果能有一些高质量标注样本做引导，模型的训练效果和泛化能力其实能提升不少。可惜的是，业内目前还没有一套高效且通用的标注工具，能把这件事完成得既快又准。从这个角度看，开发一个高效又准确的调用链标注工具，已经成了不少研发团队的迫切需求。有了高质量的数据集作为支持，相关的研究和实践才能进展更顺利。就像当年 ImageNet^[18] 对计算机视觉和深度学习带来的推动一样，一个好的数据集往往能把整个领域往前带一大步。

一般情况下，运维工程师需结合专家知识对所有调用链进行逐一标注，以获

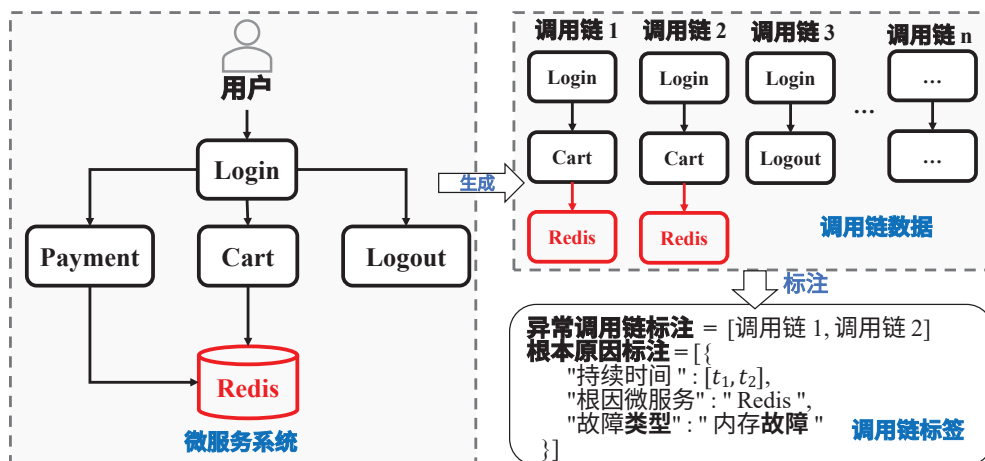


图 1.1 一个简单电子商务微服务系统的 Trace 标注流程示例。当 Redis 服务发生内存故障时，Trace 标注应包含 Redis 的异常 Trace 信息和故障根本原因信息。

取高质量的数据集。图 1.1 展示了调用链标注过程的一个简化示例。具体而言，标注过程涉及对调用链数据的深度观察，并且需要识别异常调用链并总结其故障发生时候的根因。然而，完全依赖人工对调用链数据集进行标注并不简单。传统的调用链标注方法要求运维人员逐一观察调用链的调用结构与延迟特征，并将其与相似调用链进行对比。所以，人工标注系统分析所需的调用链数据需要投入巨大的人力与时间成本。在面对大量的调用链数据以及其复杂的结构化信息时，运维工程师很难完成全量数据集的标注工作。此外，目前最先进的无监督异常检测模型在处理标注任务时表现并不理想（具体论述见第三章第五节第一小节）。因此，为了降低调用链数据人工标注的开销，本文的研究目标是构建一种半自动化的调用链标注工具，为了辅助运维工程师高效且准确地标注异常调用链及其故障根因。

第二节 国内外研究现状

一、分布式追踪技术

分布式追踪（Distributed Tracing）技术是现代微服务系统可观测性里头最核心的一环^[19]。目前工业界已经有不少成熟的方案了，比如 Jaeger^[20]、Zipkin^[21] 和 SkyWalking^[22]。它们基本都是基于 OpenTelemetry 这类标准协议来实现的，通过在请求链路里埋上 Trace ID 和 Span ID，就能把一个用户请求在各个服务之间如何调用、每一个服务处理时间是多少，完整地串联起来。其中，Jaeger 系统分布

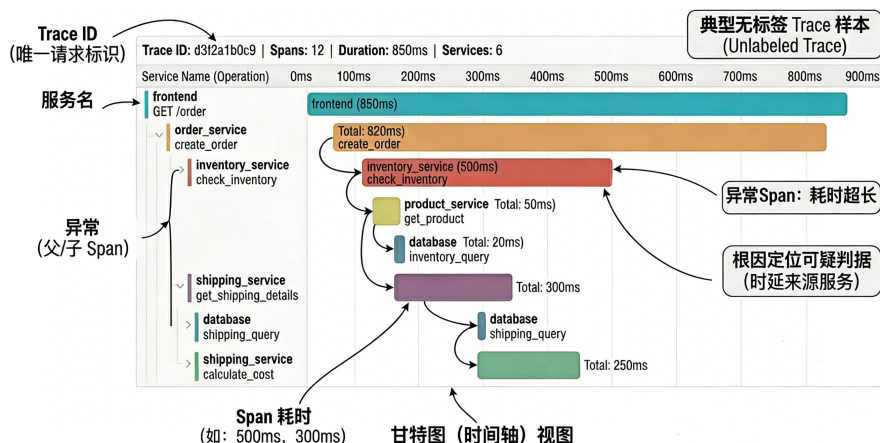


图 1.2 分布式追踪调用链可视化界面示意图

式追踪调用链可视化界面示意图如图 1.2 所示。

这些工具在日常排查故障、展示服务拓扑、分析性能瓶颈上非常完善。但在数据驱动的智能运维 (AIOps) 数据集标注背景下，它们仍面临很多关键挑战。

首先，现有追踪系统主要承担数据采集职能，其所生成的调用链仅包含结构、耗时等原始观测信息。对于依赖监督信号的机器学习方法而言，这类数据属于典型的无标签样本，模型无法从中直接获知某条调用链是否对应异常行为。

其次，标注功能的缺失是当前系统架构中的另一项结构性短板。Jaeger、Zipkin 等工具在设计上侧重于单条调用链的可视化与排查，并未提供面向海量异常数据的系统化、半自动化标注模块。面对千亿级别的 Trace 规模，完全依赖运维人员在监控界面中手工识别异常并记录真值，难以具备实际操作的可行性与效率。

上述两方面因素共同制约了下游研究的开展。由于缺乏统一的标注规范与配套的辅助工具，学术界与工业界普遍面临高质量故障数据稀缺的问题，使得许多基于深度学习的异常检测与根因分析算法在验证与迭代环节受到限制。

总的来说，现在这些追踪系统已经帮我们解决了“怎么把数据拿到手”的问题，但怎么高效地把这些海量数据转成带语义标签、能被模型理解的“语料”，还是个需要补上的短板。

二、KPI 异常标注技术研究现状

在微服务运维的其他方向，Zhao 等人^[23] 针对多变量时间序列数据提出过一个半自动标注工具，叫 Label-Less。它的思路是用一套高效、稳定的异常相似性

检索方法，帮运维人员省去在长串 KPI 曲线里反复翻找、比对异常模式的麻烦，标注效率提升了不少。

然而，尽管 Label-Less 在时序数据领域表现优异，但是它很难被使用到微服务调用链分析和标注领域，主要遇到两个问题：

- 一是数据本身不一样：KPI 数据通常表现为一维或多维的数值序列，而 Trace 数据本质上是具有复杂层级关系的有向无环图（DAG），层级关系和依赖链路都比较复杂。时序数据的相似性检索办法，很难直接套用到这种结构上。
- 二是要抓的特征不一样：Label-Less 抓的是数值模式是否相似，但调用链里很多异常其实隐藏在调用路径的异常结构里，或者是由某处延迟引发的连锁反应。为了把这些东西标注出来，工具得能理解拓扑结构才行。

可以说，KPI 数据的标注任务领域已经具有半自动化的标注工具，但调用链数据领域还缺少这么一套高效的工具。这一缺口其实也拖慢了不少可观测性研究往智能化走的速度——从这个角度看，本文研究内容既有它的学术必要，也有实际的价值。

三、基于调用链的异常检测研究现状

调用链异常检测旨在从海量的 Trace 数据中识别出执行路径异常的样本，提高系统的可靠性和稳定性。这几年，围绕调用链的异常检测，研究者们从多个维度开展了深入研究提出了很多新颖的方法。

- MultimodalTrace^[24]：把调用链里的时序信息和结构信息揉到一起。具体做法是：用 One-hot 把调用路径变成向量，再把每个 Span 的耗时也归一化后作为特征。为了减少干扰，那些很少出现的路径会被直接过滤掉。最后，利用长短期记忆网络（LSTM）对调用链中的序列依赖关系进行建模，从而实现对异常行为的精准识别。

- TraceAnomaly^[12]：采用带有后验流的高维变分贝叶斯网络对正常调用链模式进行建模。该方法引入了服务调用链向量（STV）作为数据表征，其中向量的每一维代表一条特定的调用路径，而其对应的数值则反映了该路径的响应时间。通过学习正常样本的概率分布，TraceAnomaly 能够以无监督的方式有效识别偏离基准模式的异常请求。

- CRISP^[13]：在检测模型上和 TraceAnomaly 一脉相承，但它在特征上做了更细的打磨。它通过递归找到影响端到端时延的关键路径，然后构建了一个服

务关键路径向量。这里有个巧妙的地方：它会把父 Span 的总耗时减去子 Span 的耗时，从而精准剥离出每个服务节点的净执行时间。这样一来，级联调用造成的耗时重复就被剥离了，检测时延异常也更准。

- **TraceCRL^[14]**: 借用了图对比学习的框架，把对比学习和图神经网络结合起来做表征学习。刚开始用 **DeepWalk^[25]** 给每个操作初始化向量，然后又加上了起始时间分位数、本地执行时间占比这些更细的时序特征。通过后续结合单类支持向量机（**One-Class SVM**）作为检测模型，**TraceCRL** 能够有效捕捉分布式环境下的复杂异常模式，并在异常检测任务中得到了广泛应用。

- **TraceVAE^[15]**: 它的出发点是结构和时序可能藏着不同的异常。它用图变分自编码器的思路，单独做了结构自编码器和时间自编码器，分开建模。中间还插了图注意力网络来捕捉服务节点之间的复杂关系。等学习到了正常样本的结构和时间分布，不管是调用路径异常，还是响应时间超标，都能被识别出来。

- **TraceSieve^[6]**: 它比较关注数据里的噪声问题。它用对抗训练框架搭了个自编码器，目的是把那些干扰性的噪声过滤掉，让检测更稳健。另外，考虑到系统会不断变化，模型如果每次都要重新训练就太慢了，**TraceSieve** 创新性地集成了 **VGAE-EWC** 机制，通过将变分图自编码器（**VGAE**）与弹性权重巩固（**Elastic Weight Consolidation, EWC**）相结合，在保留历史经验的同时实现了高效的增量训练。

四、基于调用链的根因定位研究现状

根因定位（**Root Cause Analysis, RCA**）是在识别异常后的深层溯源阶段，在异常被检测出来之后，还得往深挖一层，找到具体是哪个微服务或者哪个接口导致整个链路故障。针对这个问题，这几年也积累了不少方法。

- **TraceRCA^[5]**: 针对现有根因定位方法过度依赖监督信息或不切实际假设的问题，提出的一种实用的微服务根因定位框架。该方法的核心洞察在于：若某一微服务被大量异常调用链覆盖而较少被正常调用链覆盖，则其作为故障根因的可能性更高。**TraceRCA** 完整涵盖了调用链异常检测、异常微服务集合挖掘及服务排序三个阶段。

- **MicroRank**: 一种面向微服务环境中时延故障的根因定位方法，尤其适用于服务实例数量众多、依赖关系复杂的场景。其核心思路在于，通过分析正常与异常调用链中隐含的规律，为时延异常的精确定位提供支持。当系统触发异常

告警后，MicroRank 首先筛选出异常的调用链，并利用改进的 PageRank 算法对不同调用链的重要性进行加权。在此基础上，将加权后的调用链信息引入扩展的谱分析框架中，通过计算生成一份反映各服务异常程度的排序列表，从而辅助运维人员快速锁定最可能的故障来源。

- TraceRank^[26]：提出了一种结合谱分析与个性化 PageRank 的根因定位方法，用的不是聚合后的统计数据，而是细粒度的非聚合调用链数据。它会先对 Trace 做聚类，把结构相似的归到一起，再从中找出异常的。到了根因分析这一步，TraceRank 会画出一张服务调用图，用皮尔逊系数算出每个服务处理时间和前端响应时间的相关性，作为随机游走的转移权重。为了避免漏掉一些关键信息，它又用 Ochiai 公式做了轮校准，最后定位出具体是哪个服务实例出了问题。

- Sage^[27]：专门针对那种因为依赖关系复杂导致的级联故障，研发了机器学习驱动的根本分析系统 Sage。它不需要人工打标签或者使用规则，而是用无监督的因果贝叶斯网络和图变分自编码器搭了个框架，自动去学服务之间的影响关系，然后在线找出真正影响性能的根本。这个设计既考虑了诊断的准确性，也照顾到了实际落地时的可行性。

综上所述，尽管目前已涌现出大量调用链异常检测^[6,12-15,24] 和根本定位算法^[5,26-28]，但还没有研究能够实现在降低调用链数据标注工作量的同时，仍能保证获取高质量的数据集。因此，迫切需要一种专为调用链设计、能够兼顾低投入成本与高标注质量的半自动标注工具。

五、高质量数据集的启示：以 ImageNet 为例

回顾人工智能的发展，很多技术突破其实不是靠某个算法一蹴而就的，背后往往少不了一套高质量的数据集。计算机视觉里的 ImageNet^[18] 就是个典型例子。

这个数据集是李飞飞团队逐一标记出来的。它按照 WordNet 的语义层级整理，有超过 1400 万张图，分了 2 万多个类别，而且全是人工标注的。它厉害的地方不只是“大”，还有两点很关键：

- 首先是标注的质量与一致性。当年靠 Amazon Mechanical Turk 众包平台，一张一张图片进行标注，具有很高的一致性。后来的 AlexNet、ResNet 这些模型能跑起来，很大程度上是依赖了这批数据集的质量。

- 其次是它成了行业的“度量衡”。ILSVRC 挑战赛持续了很多年，计算机

图像领域用同一套标准比效果，算法进步变得可量化、可比较。

不过回过头看，ImageNet 的成功也让人忍不住反思：调用链数据分析领域，有没有属于自己的 ImageNet？

在微服务运维和分布式追踪这个方向上，情况就不那么乐观了，数据稀缺的问题相当突出。和视觉领域对比来看，这给我们的研究带来几点思考：

- 一是从“靠算法”到“靠数据”的转变。视觉研究早期大家还在琢磨怎么设计更好的特征，现在已经是数据主导的时代。但在调用链分析这块，由于标注成本太高，很多方法还是停留在规则判断或者无监督学习上，遇到复杂故障就容易露怯。

- 二是标注的成本很高。ImageNet 启示“更多更好的标注 = 更强的模型”，但调用链的标注比图像分类麻烦得多。图像只需要标注人员识别出猫和狗等具体的图片，而调用链的异常判断，往往需要资深运维工程师翻看拓扑、排查指标，依赖专家经验，而不是大众的常识。

- 三是缺一个公认的“黄金标准”数据集。现在缺乏像 ImageNet 那样的权威数据集，被学界和工业界一起接受、一起使用。

第三节 研究内容及主要贡献

本文的研究内容及主要贡献分为两个方面。一方面是基于调用链数据的半自动化异常标注机制；另一方面是基于调用链数据及微服务拓扑结构的根本原因定位辅助定位和标注。本文的研究内容组织如图 1.3 所示。

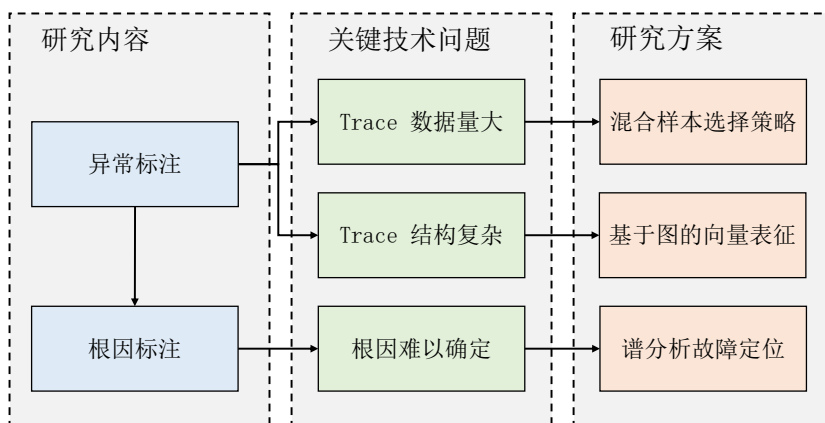


图 1.3 本文研究内容组织

本文的目标是设计一个半自动化的调用链辅助标注工具，不仅能够对调用链数据进行异常标注，也能够对微服务系统的故障根本原因进行辅助标注，减少人工标注的成本和开销。在构建该工具的过程中，主要面临以下三大挑战：

1. Trace 的数据量巨大。微服务系统每天能够大量的 Trace 数据用来打标。比如，在 eBay 的基础设施微服务系统运行中，一天能够产生 1500 亿条左右的 Trace 数据^[29]。全都由人工手动打标完的是不可能的。

2. Trace 的数据结构很复杂。Trace 有一个比较突出的特点就是复杂且丰富的结构特征^[30]。因此，如何比较好地对 Trace 数据结构进行建模成为了一个难题。由于服务间的层级调用关系，以及并行和异步调用机制的存在^[10]，Trace 通常呈现复杂的图结构。一般来说，一个 Trace 数据集包含的服务操作数量往往在数十到数百个之间^[14]。

3. 根本原因难以定位。再 Trace 数据集的异常标签都打标完成之后，对根本原因的标注始终成为一个挑战。通常来说，根因标注需要考虑一段时间内的所有调用链请求数据。但是对于人来说，去检查所有的异常 Trace 数据并且总结根因信息是非常耗时费力的。

为了减轻标注调用链数据集的工作量，并且确保高质量的调用链数据集做异常检测和根因定位等工作，解决这些标注困难的挑战是很有必要的。再本研究中，我们提出了 LabelEase，这是一个在微服务系统中高效且准确的半自动化调用链标注工具。为了解决第一个挑战，我们使用主动学习的方法，运维工程师只需要标注一小部分的调用链数据就可以得到一个高质量的调用链数据集。在完成所有调用链数据的异常标注工作之后，我们聚合异常的调用链数据得到故障根因区间，并且提供可视化的信息参考帮助运维工程师进行根因的标注。为了解决第二个挑战，我们训练了一个图神经网络去对所有调用链数据进行编码。基于图对比学习的方法，最终确保相似的调用链数据能够在向量空间里能够被表示成相似的特征向量。为了解决第三个挑战，我们设计了调用链聚合和根因定位的方法去整合异常的调用链数据识别微服务系统的故障区间时间段去辅助运维工程师进行根因的标注。本文的核心贡献总结如下：

1. 据我们所知，本文是第一个针对调用链数据标注工作减少标注工作量的工作研究。本文结合高效的算法和运维工程师的专家经验，帮助运维工程师高效并且准确地标注调用链数据集。

2. 本文创新性地提出了 LabelEase，它由三个部分组成：基于图的调用链表

征模块，异常标注模块和根因标注模块。LabelEase 对大量的调用链数据进行聚类得到几个不同的簇。然后挑选最具有代表性的调用链数据给运维工程师进行标注，极大地减少了标注的工作量。

3. 本文使用两个数据集进行了一系列实验去评估了 LabelEase 的性能和效果。实验结果表明我们的方法在微服务系统调用链数据集的标注工作上，仅需运维工程师标注一小部分调用链数据，就可以分别实现了 0.99 和 0.98 的 F_1 -score。

4. 本文发布了一个高质量的调用链数据集。它不仅能够标识出每一条调用链是正常还是异常，也能够展示发生故障时微服务系统的根因是什么。另外，我们已经将本文 LabelEase 的源代码和打标好的调用链数据集进行了开源¹，帮助读者更好地理解我们的工作。

第四节 论文组织结构

本文共由五个章节组成，各个章节的内容概述如下：

第一章，绪论。该章节首先介绍了本文的研究背景与研究意义，接着概述了本文研究内容相关的国内外研究现状，最后介绍了本文研究的核心内容及主要贡献，并对本文的组织结构进行了说明。

第二章，相关基础知识。该章节详细阐述了与本文相关的背景知识，首先介绍了微服务系统和调用链数据的相关定义，接着介绍微服务系统运维中两大核心内容异常检测和根因定位，然后介绍了本文两大研究内容所需要使用的相關基础知识，最后介绍实现标注工具所用到的前后端框架。

第三章，基于调用链数据的半自动化异常标注机制。该章节首先分析了调用链标注的重要性和必要性；接下来详细介绍了半自动化的调用链数据标注工具 LabelEase 的方法和原理。然后介绍最终实现的调用链异常标注工具的使用；之后对调用链标注任务使用的数据集、对比方法、评估指标和实验运行环境进行了全面的介绍；最后将 LabelEase 和多个无监督异常检测算法进行对比，对不同模块进行消融实验，还进一步分析了标注数量对数据集质量的影响程度分析，从而证明本文提出的方法的有效性和实用性。

第四章，基于调用链数据的微服务故障根因辅助标注机制。该章节首先分析了基于微服务系统的根因标注的重要性；然后详细介绍了 LabelEase 在微服务根因标注时的方法原理；接下来介绍了 LabelEase 标注微服务根因时的工具实现

¹<https://doi.org/10.5281/zenodo.13338156>

和使用；最后通过实验进一步证实了 LabelEase 根因标注的准确性，并分析不同的故障表现在 **Trace** 上的表现的不同。

第五章，总结与展望。该章节对本文提出的两项工作进行总结，并对下一步可能的研究方向进行了展望。

第二章 相关基础知识

随着微服务系统的升级和大规模部署，系统的复杂性和可靠性越来越受到关注，因此维护微服务系统的稳定性和可用性的意义越来越大。为了维护好微服务系统的正常运行和服务管理，微服务系统产生的调用链数据毫无意外成为最重要的数据来源之一。为支持更多基于调用链数据的算法和模型提供高质量的训练和评估的数据集，本文计划实现一个半自动化的调用链标注工具，减少人工标注投入的成本。本章详细介绍微服务系统和调用链的基础背景，微服务系统的核心运维任务，以及半自动标注的的相关基础知识，为第三章和第四章的机制设计与实现提供必要的背景支持。

第一节 微服务系统架构概述

微服务架构是一种将复杂单体应用拆解为多个小型、自治且松耦合服务单元的分布式系统设计范式^[31]。与传统的单体架构不同，微服务架构强调业务功能的原子性，通过将多个复杂的系统功能划分为可扩展独立运行的服务进程，实现了开发、部署与运维各环节的职责解耦^[32]。微服务架构因其在可扩展性、灵活性、可维护性和资源高效利用等方面的显著优势^[33-34]，在互联网产品实现选用架构里越来越受欢迎。

一、核心技术特征

微服务系统的优越性主要体现在其高度的自治性与灵活性上，其核心特征包括：

- 单一职责与服务粒度：每个微服务聚焦于系统内某一特定业务能力，体现了单一职责原则^[35]。以电商系统为例，身份验证、库存管理和订单处理通常由独立的服务分别承担。这种细粒度的划分方式有助于保持代码库的可维护性，并降低功能演进过程中的耦合程度。
- 技术异构性与独立部署：微服务架构支持不同服务根据自身需求选用合适的技术栈，包括编程语言、数据库或中间件。各服务拥有独立的开发与部署生命周期，支持在不影响系统整体运行的前提下进行更新或扩展，从而提升了持续集成与持续交付（CI/CD）的灵活性^[36]。

• 去中心化数据管理：为保障服务间的真正解耦，微服务常采用“一服务一数据库”的数据隔离策略。例如，在如图 2.1 所示的系统中，“登录”服务可能采用关系型数据库维护用户信息，而“购物车”服务则更适合使用 Redis 这类缓存系统以应对高并发读写场景。该设计有助于避免底层存储层面的资源竞争，增强系统的整体稳定性。

二、服务间通信与交互机制

在微服务环境中，由于服务分布在不同的物理或虚拟节点上，跨进程通信（Inter-Process Communication, IPC）成为系统协同工作的关键。

服务之间具有依赖关系，通常通过定义明确的轻量级协议构建通信渠道。常见的交互模式包括基于 HTTP/JSON 的 RESTful API、高性能的 gRPC 以及基于消息中间件的异步通信^[37]。以图 2.1 所示的业务流为例：当用户发起购买请求时，“支付（Payment）”服务需同步调用“钱包（Wallet）”服务进行余额校验，同时可能通过异步方式通知“订单”服务更新状态。这种基于 API 的无缝集成不仅赋能组织实现按需扩容，也使其能够迅速响应不断演变的业务需求与技术变革^[38]。

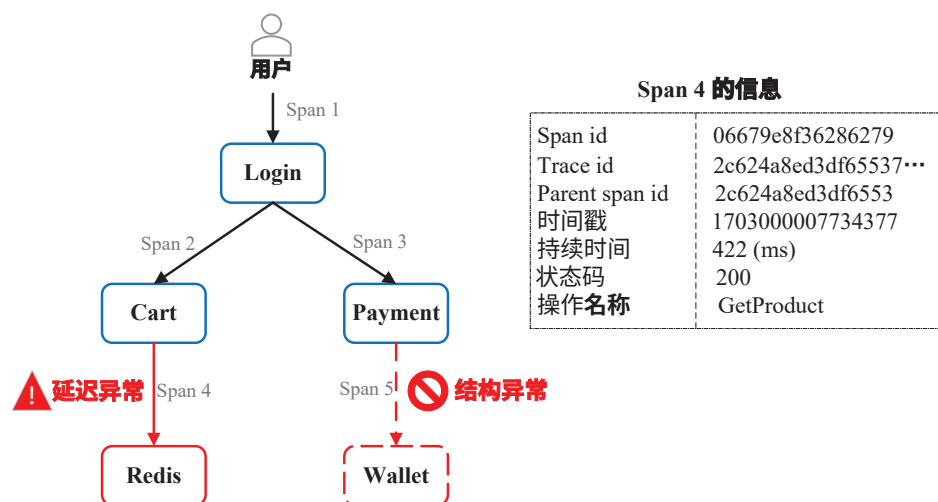


图 2.1 微服务调用链结构示例图。图中展示了请求在不同微服务单元（如 Login, Cart, Redis）间的传递过程。

第二节 调用链

调用链（Trace）描述的是一个请求在微服务系统里经过的完整路径^[39]。一个用户请求从前端发起后，往往需要多个微服务协同处理才能完成。调用链正是贯穿这一过程的一条完整记录，它追踪请求在各个服务之间的流转路径，记录了每一次调用的发起方与接收方、响应时间、执行状态等关键信息。可以将其理解为系统内部的一次“行程单”，通过结构化方式呈现数据流动与业务逻辑的执行过程。借助调用链，运维人员能够直观掌握服务间的调用关系、性能表现以及潜在的故障节点，从而更有效地进行系统诊断与优化，并提高系统稳定性^[29,40]。此外，基于历史调用链的分析能够揭示长期的性能趋势和异常模式，为系统优化和未来架构改进提供关键的参考和支持^[10]。

图 2.1 展示了一个由 Span 1 到 Span 5 组成的调用链例子。每个 Span 代表一个具体操作^[29]，比如一个服务的入口或一次数据库查询。每个 Span 里都会包含一些元数据：这次调用属于哪个请求（Trace ID）、它的父节点是谁、调用的服务和方法名，以及开始和结束的时间戳（精确到微秒）。通过这些信息，我们可以算出每个节点的耗时（Duration），也能还原出服务之间是串行调用还是并发执行。

把这些 Span 拼起来，调用链可以用两种视角呈现：一种是时间线视图（像甘特图那样），看清每个环节花多长时间；另一种是拓扑视图，看清服务间怎么连接的。这两种视图不仅能帮我们还原故障现场，也为后面要讲的半自动化标注工具提供了原始数据支撑——有了调用链的时空信息，我们可以更准确地识别异常，降低专家的手动标注负担。

调用链的异常一般可以分成两类：

- 延迟异常：也就是某个节点耗时异常变长，通常是因为网络卡顿或延迟、服务负载高了（比如 CPU 或内存占满）。
- 结构异常：指的是调用路径和预期不符，比如本来该有的调用请求中断，可能由网络丢包、配置错误或者代码逻辑错误引起。

以图 2.1 为例，若“购物车”服务调用 Redis 的过程中，Span 4 的耗时显著超出正常范围，则属于延迟异常；又如，若“支付”服务本该调用“钱包”服务的调用关系缺失，则反映了结构异常。

第三节 异常检测与根因分析：微服务运维的双重范式

在分布式微服务架构中，系统故障的演进通常表现为从性能退化到服务崩溃的复杂过程，严重影响系统服务的正常运行^[41]。为了实现高效的系统自愈与运维保障，业界通常将故障处理流程解构为相互依存的两个阶段：异常检测与根因分析。

一、异常检测

异常检测（Anomaly Detection），也被称为孤立点检测，是数据挖掘与机器学习领域的核心研究方向^[42]。其基本定义为识别数据集中与绝大多数观测值不符、或不遵循预期模式的少数样本。在工业系统与大型互联网软件架构中，异常通常代表了系统运行过程中的故障隐患、性能瓶颈或安全威胁^[43]。

为了确保微服务系统的平稳运行和高效管理，维护服务质量和提升性能，基于调用链的异常检测是必不可少的^[8]。在微服务架构下，异常检测已经不只是关注单个指标，而是要对整个请求链路的交互行为做深入分析。它是可观测体系里的第一道防线，目标就是从海量的调用链数据里，实时找出那些偏离正常模式的请求^[44]。

跟传统的日志监控或者指标告警比起来，基于调用链的检测要细得多，主要分为三种类型：

- 结构异常检测：重点关注调用路径有没有异常。比如代码异常、配置错误，调用链的路径可能就少了几个节点，或者多了一些不该有的分支。通过给调用链建模成有向无环图，我们能捕获到像死循环、不该发生的重试，或者某些节点被悄悄绕过这类结构性故障。

- 性能（时延）异常检测：侧重于对 Span 响应时间及整体链路延迟的波动建模。在分布式环境下，时延异常往往具有级联效应（Cascading Effect），即下游服务的轻微抖动可能导致上游服务出现大面积的服务质量（QoS）违规。基于调用链的检测能够精准剥离各节点的净执行时间，识别出真正的性能瓶颈。

- 语义与属性异常检测：结合 Span 里带的状态码、标签或者业务日志，发现像“连不上数据库”、“权限校验失败”这种业务层面的报错。

基于调用链的异常检测机制不仅能够实时触发告警，同时捕获异常发生时的完整上下文信息，为后续的故障回溯与分析提供支撑，以促进问题的解决修复，进而防止服务中断或性能下降^[12]。作为保障系统稳定性的首要环节，该机

制也为后续的标注工作筛选出可疑样本，从而构成智能化运维流程中不可或缺的基础支撑。

二、根因分析

在当今快速发展的微服务平台中，系统故障可能对业务运营产生深远影响，导致长时间停机、数据丢失和财务损失^[45-47]。准确且高效的故障根因定位已成为有效软件管理和运营弹性的关键组成部分^[48-49]，成为了人工智能运维领域的一个重点。在微服务运维里，根因分析是从“知道出事了”到“搞清楚哪出事了”的关键一步。异常检测负责告警——告知系统现在出现问题；而根因分析是在故障发生之后，洞察数据表现，找到真正引发故障的那个源头^[50]。

异常检测与根因分析在分析维度和目标指向上存在本质区别。异常检测的核心任务在于“状态判别”，即通过时序建模或统计方法，在海量观测数据中识别出偏离正常模式的行为，从而回答系统“是否发生异常”这一问题。相比之下，根因分析则聚焦于“因果溯源”，其目标是解释“为何发生异常”。这一过程始于观察到的系统级症状，并系统地缩小调查范围以确定根本原因^[51-52]。在微服务架构中，由于服务间存在复杂的调用依赖^[53]，某一节点的轻微异常往往经由紧耦合的链路迅速扩散，引发下游大量服务出现虚假的异常表现^[54]。这种级联效应使得故障在观测层面呈现出多点并发的表象，而根因分析的任务正是从这些表象中剥离干扰，还原故障传播路径，实现从现象到本质的逆向定位。

要实现这个目标，一般依赖几方面的信息互相印证：

- 拓扑关联分析：先构建服务之间的依赖图，再沿着图里边的方向去推导，看异常是如何传递的，哪一个服务调用次数多，分清楚哪些服务是“受害者”，哪些服务节点才是根因。

- 时序演化追溯：基于时间线观察。如果一个服务先引发故障，其他服务被影响也出现异常，那最先发生异常的服务节点嫌疑最大。通过比对各个节点的时序偏移，能锁定最先偏离基线的那一个微服务节点。

- 跨维度因果推断：仅关注调用链还不够，还需要结合日志、资源指标（比如 CPU 飙高、磁盘 I/O 卡顿）一起分析。有时候根因隐藏在底层基础设施里，比如某台机器的物理故障，导致运行在上面的业务层全受到影响。

根因定位不仅是一项技术诊断环节，更是运维决策的关键依据。该过程不仅要求识别出具体的物理节点或逻辑微服务，还需要结合专家知识库或自动化

推理模型，研判故障传播的深度、影响范围及其对业务 SLA（服务等级协议）的冲击程度。通过将排查视野聚焦于导致整体系统效能下降的“原始触发点”，根因分析能够生成针对性的修复建议，如服务熔断、资源扩容或配置回滚。

总而言之，根因分析是整个故障处理闭环里最重要的一环。它不仅缩短了故障修复时间（MTTR），也为系统的长期优化积累经验。这种从现象到本质的诊断闭环，为本研究提出的 LabelEase 框架中关于根因标签的自动推荐与专家反馈机制奠定了理论基础。

第四节 异常标注中的相关基础知识

一、主动学习

开发调用链异常标注工具对于运维领域获取高质量异常检测数据集至关重要。然而，尽管样本标注是实现当前智能 AI 模型的重要起点^[55]，但受多种复杂因素影响，数据标注过程并非易事。幸运的是，主动学习（Active Learning）为减轻标注工作量提供了一个理想的解决方案。

主动学习是一种旨在最小化标注样本数量的同时实现模型性能增益最大化的方法^[56]。其核心目标是从未标注的数据集中筛选出最具“信息量”的样本，并将其提交给领域专家进行标注。通过这种方式，主动学习能够在保持算法性能的同时，显著降低标注成本。

与侧重于设计高性能特征提取模型的深度学习（Deep Learning）不同，主动学习主要关注基于数据集的处理方法。它通过设计精细的查询规则，从未标注数据集中挑选出最优样本，从而最小化标注代价。查询规则的设计对主动学习的性能具有决定性影响，常见的策略包括基于不确定性（Uncertainty-based）的方法、基于多样性（Diversity-based）的方法以及预期模型变化（Expected Model Change）方法。

二、图神经网络与表征学习基础

（一）图神经网络基本原理

图神经网络（Graph Neural Networks, GNN）是一种专门用于处理图结构数据的深度学习框架^[57-58]。与在欧几里得空间（如图像）上运行的传统卷积神经网络（CNN）不同，GNN 通过利用图的拓扑结构来捕获非欧几里得空间中的依

赖关系。

GNN 的核心机制是消息传递（Message Passing）或邻域聚合（Neighborhood Aggregation）范式。对于图 $G = (V, E)$ 中的节点 v ，其隐藏状态表示 h_v 通过聚合其局部邻居的特征进行更新。形式上，GNN 的第 k 层可以描述为：

$$m_v^{(k)} = \text{AGGREGATE}^{(k)}(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\}) \quad (2.1)$$

$$h_v^{(k)} = \text{UPDATE}^{(k)}(h_v^{(k-1)}, m_v^{(k)}) \quad (2.2)$$

其中， $h_v^{(k)}$ 表示节点 v 在第 k 层的嵌入向量， $\mathcal{N}(v)$ 表示节点 v 的邻居集合，AGGREGATE 是一个可微且具有排列不变性的函数（如求和、均值或最大池化）。通过多层聚合，每个节点的嵌入向量能够捕获更大感受野内的结构信息。

（二）基于 GNN 的调用链表征学习

表征学习旨在将原始的高维稀疏数据映射到低维稠密向量空间（即嵌入，Embedding），同时保留其核心的语义和结构属性。在微服务调用链场景下，基于 GNN 的表征学习具有显著优势：

- **结构编码能力：**调用链天然具备有向无环图结构，其中包含了服务间的层级依赖与并行执行关系。图神经网络能够对这种结构进行显式建模，有效捕获序列模型难以保留的拓扑信息。

- **多源特征融合：**GNN 允许将节点级属性（如 Span 的持续时间、操作名称、状态码等）与全局图拓扑结构进行无缝集成。这使得生成的嵌入向量对性能指标波动和结构变化都具有高度的敏感性。

- **相似性保持：**通过对比损失等训练目标，GNN 能够使具有相似调用模式或故障特征的调用链在嵌入空间中彼此靠近，为后续聚类或检索任务提供支持。

（三）典型图神经网络模型分类

随着图深度学习理论的不不断演进，针对不同结构特征与任务需求，学术界提出了多种经典的 GNN 变体模型。在微服务调用链分析领域，以下几种模型具有代表性：

- **图卷积网络 (Graph Convolutional Networks, GCN)^[59]：**GCN 把图像里的卷积操作运用于图数据，通过拉普拉斯矩阵对邻居节点做一阶近似。每一层更新节点的时候，就把邻居的特征进行加权求和（按度归一化），然后进行线性变换。

在调用链数据分析中，GCN 能捕捉到服务之间的空间依赖关系，计算量也不大。但它有个问题——对所有邻域一视同仁，分不清哪个调用更重要，这在有些场景下就不够灵活。

- 图注意力网络 (Graph Attention Networks, GAT)^[60]: 为了解决 GCN 无法动态分配邻域权重的问题，GAT 引入了注意力机制 (Attention Mechanism)。GAT 通过学习节点与其邻居之间的相对重要性 (即注意力系数)，实现对邻域信息的自适应加权聚合。在处理微服务调用链时，GAT 具有显著优势：它能够识别出调用路径中的关键路径或故障传播的核心节点，例如在复杂的并发调用中，GAT 可以自动为高延迟或异常状态的服务节点分配更高的权重，从而生成更具判别力的调用链表征。

- 图同构网络 (Graph Isomorphism Network, GIN)^[61]: GIN 旨在解决传统 GNN 在区分不同图拓扑结构能力上的瓶颈。通过证明聚合函数必须是单射的 (Injective)，GIN 用多层感知机加求和来做聚合，理论上能达到 Weisfeiler-Lehman (WL) 图同构测试的上限。对于微服务系统来说，当系统拓扑结构发生细微变化，或者两种调用模式长得很像但行为不同的时候，GIN 能更敏锐地捕捉到结构上的异常。

- 门控图神经网络 (Gated Graph Neural Networks, GGNN)^[62]: GGNN 将门控循环单元 (GRU) 引入消息传递过程，通过时间步迭代来更新节点状态。这种模型特别适用于具有序列性或强逻辑流向的图数据。考虑到调用链本质上是具有严格时序先后顺序的有向无环图，GGNN 能够有效建模服务调用在时间轴上的演化逻辑，捕捉跨越多个跳转层级的长距离依赖关系。

三、聚类

聚类 (Clustering) 是无监督学习领域的核心技术之一^[63]，其目标是将一组无标签的样本根据其内在相似性划分为若干个不相交的子集，每个子集被称为一个“簇”。在同一簇内的样本之间具有较高的相似度，而不同簇间的样本则存在显著差异。

聚类过程本质上是基于特征空间中的距离度量或密度分布进行的数据划分。常见的距离度量方式包括欧几里得距离 (Euclidean Distance)、曼哈顿距离 (Manhattan Distance) 以及在处理高维嵌入向量时常用的余弦相似度 (Cosine Similarity)。

对于微服务调用链而言，聚类分析能够自动发现具有相似执行路径或时延特征的请求序列。给定一个调用链表征向量集合 $\mathcal{H} = \{h_1, h_2, \dots, h_n\}$ ，聚类算法旨在寻找到一种映射关系，使得簇内离散度最小化，从而实现对系统运行状态的有效抽象。

根据划分策略的不同，聚类算法主要分为以下几类：基于划分的方法：

- 以 K-Means 为代表，通过预设簇的数量 k ，迭代地将样本分配到距离最近的质心。该算法计算效率高，适用于处理大规模的结构化调用链特征向量。
- 基于密度的方法：以 DBSCAN 为代表，该算法通过识别特征空间中的高密度区域来形成簇。其优势在于能够识别任意形状的簇，并有效过滤掉噪声点（即离群值），这在识别罕见故障模式时具有重要价值。
- 基于层次的方法：层次聚类通过构建聚类树实现样本的逐层聚合或分裂，能够展现数据在不同粒度下的相似结构。

在本文研究中，聚类分析发挥着关键的作用。通过对海量待标注调用链进行聚类，标注工具可以从每个簇中选择最靠近质心的代表性样本推荐给专家。这避免了专家对大量同质化异常进行重复标注，极大地提升了标注效率。

第五节 Django 后端开发框架

本文后端系统采用 Python 语言编写的 Web 框架 Django 进行构建。Django 遵循“自带电池”（Batteries-included）的设计理念，旨在通过高度集成的组件加速复杂数据库驱动型网站的开发进程。在微服务运维工具的研发中，Django 提供了稳健的并发处理能力与灵活的数据接口支撑。

一、MVT 架构模式

与传统的 MVC（Model-View-Controller）模式类似，Django 采用了 MVT（Model-View-Template）架构模式，实现了业务逻辑、数据模型与用户界面的深度解耦：

- 模型层：负责数据结构的定义与数据的存取逻辑。通过内置的 ORM（对象关系映射）机制，开发者可以使用 Python 类定义数据库表结构，从而避免编写复杂的原生 SQL 语句，极大地提升了调用链等结构化数据的管理效率。
- 视图层：作为系统的核心逻辑处理单元，负责接收前端请求、调用业务算法（如主动学习策略或图表征模型）并返回响应结果。

- 模板层：负责结果的呈现逻辑。在 LabelEase 这种前后端分离的架构中，Django 主要充当 RESTful API 服务端，通过 JSON 格式与前端可视化界面进行高效通信。

二、在 AIOps 科研工具中的适用性

选择 Django 作为本研究后端框架的主要原因包括：

- 高效的数据持久化：其强大的 ORM 机制能够轻松应对大规模 Trace 数据的存储、筛选与异常区间聚合任务。
- 系统稳健性：Django 成熟的中间件（Middleware）机制与自动化管理后台（Admin），为标注数据集的真值（Ground Truth）导出与系统监控提供了极大的便利。

第六节 Vue.js 前端开发框架

本文的前端界面基于 Vue.js 框架进行构建。Vue.js 是一款用于构建用户界面的渐进式 JavaScript 框架，其核心采用自底向上增量开发的设计。在微服务运维标注工具中，由于需要频繁处理海量的图结构数据渲染与复杂的专家交互反馈，Vue.js 凭借其轻量级架构、高效的响应式系统以及组件化开发范式，成为了构建高性能标注工作台的理想选择。

一、MVVM 模式与响应式原理

Vue.js 的核心架构基于 MVVM（Model-View-ViewModel）模式，实现了数据模型与视图层的深度解耦：

- 模型层：对应后端 Django 提供的 RESTful 接口数据，包括 Trace 拓扑结构、Span 耗时及算法推荐的异常得分。
- 视图层：负责将调用链数据转化为直观的拓扑图、时序 Gantt 图及根因排序列表。
- 视图模型层：作为 Vue.js 的核心，利用其响应式数据绑定机制，实时监听模型层的变化。

当运维专家在界面上点击某个节点进行标注或调整时间轴时，视图模型层会自动触发视图的局部更新，而无需刷新整个页面。这种高效的更新机制对于处理包含数千个 Span 的大规模调用链数据集至关重要。

二、在调用链标注任务中的优势

相较于传统的前端框架，Vue.js 在 LabelEase 系统中的应用优势体现在：

- 极佳的用户体验：通过虚拟 DOM（Virtual DOM）技术，在专家频繁缩放拓扑图或切换 Trace 样本时，保持毫秒级的交互响应。
- 易于集成可视化库：其生命周期钩子（Lifecycle Hooks）能够完美对接高性能可视化库，实现了“数据变化驱动图形重绘”的直观标注体验。
- 开发效率提升：简洁的模板语法与声明式渲染，使得科研人员能将更多精力集中于标注逻辑与算法反馈的实现。

第七节 本章小结

本章节对于本文中提到的相关基础知识进行了一系列的介绍，针对微服务系统的相关概念、调用链数据相关概念、主动学习的相关基础知识以及前后端实现的主要框架等内容进行了简单的解释说明。了解本章的内容便于阅读第三章和第四章内容的时候更好地理解相关机制的设计和实现原理。

第三章 基于图表征与主动学习的异常标注

微服务故障往往会引发连锁反应，对用户体验及业务营收造成严重损害。为保障微服务系统的稳定性，基于调用链数据的异常检测技术已成为工业界与学术界的关注焦点。随着机器学习技术的飞速发展，利用驱动模型自动识别调用链异常，已成为提升大规模分布式系统可观测性的关键研究趋势。然而，现在普遍缺少高质量的调用链数据集。由于调用链数据量非常大且结构特别复杂，导致人工标注的成本和开销过大。因此，我们提出 LabelEase，一个半自动化的调用链辅助标注工具。LabelEase 基于图表征和主动学习，运维工程师只需要标注少部分样本，就可以得到一个高质量的数据集，极大提高了标注效率和减少了标注成本。实验结果表明，LabelEase 在准确率和效率方面均高于现有调用链异常检测方法。

第一节 引言

在微服务运维领域，面对海量调用链（Trace）数据，完全依赖人工标注获取高质量真值（Ground Truth）已被公认为“具有不可行性”。这种不可行性主要源于数据规模带来的极限挑战以及数据结构复杂性导致的认知负荷瓶颈。

海量数据量与人工吞吐量的极度失衡微服务架构下，数据生成的速率与人工处理的效率之间存在着量级上的鸿沟：千亿级的生成速率：在大型互联网基础设施（如 eBay、阿里巴巴）中，微服务系统每日生成的 Trace 数量可达千亿级别（约 10^{11} 条）。即便在发生故障的分钟级窗口内，产生的相关 Trace 往往也数以万计。人工标注的边际成本溢出：假设一名资深专家标注一条复杂 Trace 平均耗时 1 分钟，如果对所有调用链数据进行标注，其所需的人力投入也将超出现有运维团队的承载极限。实时性与滞后性的矛盾：系统故障的处置要求分钟级响应，而人工标注的滞后性决定了其无法为实时的异常检测和根因定位模型提供即时的反馈驱动。

高维图结构与人类认知负荷的矛盾调用链并非简单的线性日志，其复杂的拓扑属性使得标注过程极度依赖高门槛的领域专家知识：拓扑结构的复杂性：Trace 本质上是有向无环图，单条 Trace 可能包含数百个 Span，涉及数十个微服务的深层嵌套调用。人工标注者需要通过 Trace 日志不断进行父子节点的跳转、

耗时对比和状态码对齐，这构成了极高的空间认知负荷。异步与并发带来的干扰：现代微服务大量采用异步调用和并发处理机制。人工在审视调用链时，难以直观地从时间交错的 Span 中分辨出真正的因果依赖关系，导致标注的准确率随 Trace 复杂度的增加而急剧下降。根因判定的不确定性：故障往往具有级联效应，表象异常（如上游服务延迟过高）与真实根因（如底层 Redis 内存饱和）在空间上可能相隔多个层级。专家需要综合分析一段时间内的多条 Trace 模式才能做出判定。这种从“海量模式”中提取“共性特征”的任务，人类大脑的并行处理能力远逊于机器学习模型。

综上所述，人工标注不可能性带来的必然路径由于上述两个维度的限制，传统的人工标注面临以下科学难题：标注的一致性难以保障：不同专家或同一专家在疲劳状态下，对复杂拓扑结构的判定标准难以保持统一，导致数据集存在噪声。标注的覆盖率极低：人工只能覆盖常见的故障类型，对于长尾分布中的罕见异常，由于样本量巨大且难以通过肉眼检索，往往会被遗漏。因此，人工标注的不可行性直接倒逼了技术路线的演进——即从“全人工标注”转向“基于主动学习的半自动辅助标注”。通过图神经网络对复杂结构进行自动化特征表征，再结合主动学习筛选出最具代表性的极少数样本交由专家决策，是解决海量数据背景下高质量数据集获取难题的唯一科学路径。

第二节 方法介绍

一、整体流程概述

LabelEase 的总体框架如图 3.1 所示。它由三个核心模块组成：基于图的调用链表征模块，异常标注模块和根因标注模块。基于图的调用链表征模块，异常标注模块和根因标注模块。LabelEase 能够帮助运维工程师针对原始调用链数据进行异常标注和微服务根因的标注，最终得到一个高质量的调用链数据集。

在基于图的调用链表征模块中，首先，无标签的调用链数据会被运维工程师进行上传并被预处理成图结构的数据。然后，我们基于不同调用链的特征训练了一个图神经网络模型去区分不同调用链数据之间的差异。最后，我们使用训练好的图神经网络模型去对所有的调用链数据进行编码，输出它们在多维空间中的向量表征。

在异常标注模块中，我们采用一种混合代表性选择（Hybrid Representative

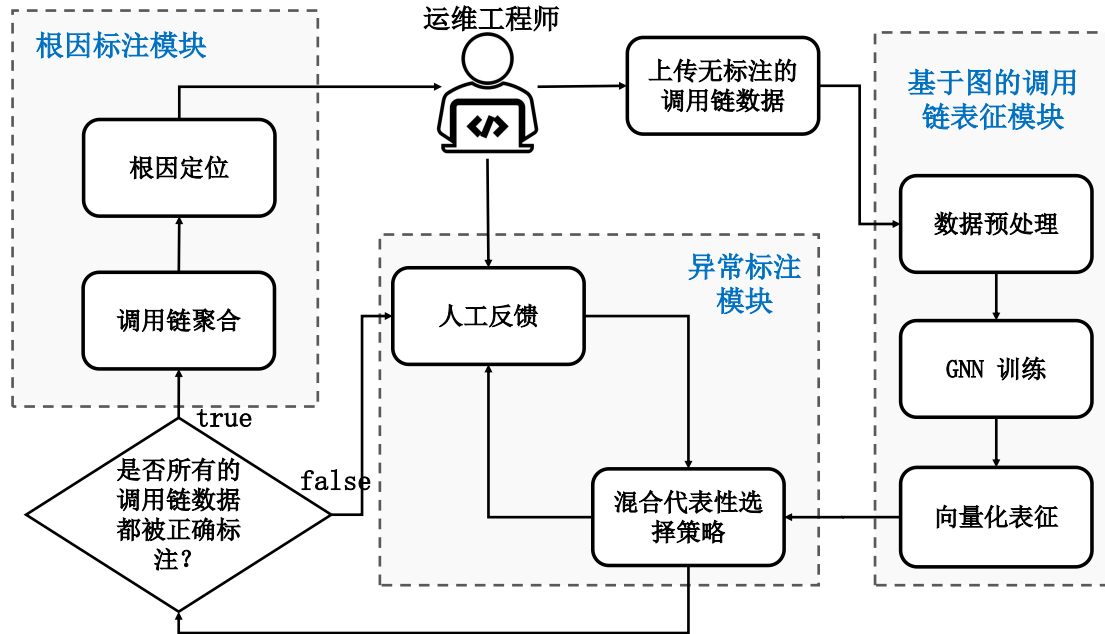


图 3.1 LabelEase 框架

Selection) 方法，将调用链表征向量聚类为多个类别。运维人员仅需发挥专业经验，对每个聚类的中心点样本进行异常识别。通过引入人工反馈，系统利用主动学习机制自动且准确地完成剩余海量调用链的标注工作。

在根因标注模块中，系统会自动聚合异常区间内的调用链。运维人员通过分析各服务的访问频率并审查相关的调用依赖关系，能够更精准、高效地锁定系统故障的根本原因。

本章仅针对 Trace 的异常标注的前两个模块进行详细介绍，根因标注模块具体将会在第四章进行介绍。

二、基于图的调用链表征

(一) 数据预处理

在调用链的分析过程中，我们主要关注于调用链的语义信息（比如微服务的名字和操作的名称），时间特征，Span 的状态码信息。以及不同微服务之间的调用关系即调用链的结构特征。参考 Zhang 等人^[64]提出的方法，我们将原始的调用链数据转换为一个个图数据结构的格式。

对于每一个 Span 的语义信息的提取，我们使用在微服务系统中常见分隔符号（比如 ‘/’, ‘-’, ‘:’）去划分涉及到文本内容。所有的文本会被转换为小写。并

移除数字及标点符号等非字符串。对于词表外（OOV）词汇或新的复合词，我们使用 WordPiece 分词技术^[65]，通过将单词拆分为基础字符并迭代识别最常见的字符组合，直至达到预设词表规模。接下来，我们使用一个预训练的 BERT 模型^[66]的特征提取的能力对文本列表进行编码，获取语义向量，以表示每个 Span 的微服务名字和操作名称。

对于每一个 Span 的时间特征的提取，服务调用的开始时间与持续时间为分析微服务系统的性能与行为提供了重要依据。通过分析每个 Span 的持续时间，等待时间，当前执行时间和相对开始时间^[64]，我们可以全面理解微服务系统的执行和表现情况。一方面之前进行语义编码的高维向量表示会导致 Span 时间特征的表征向量被忽略。另外一方面，不同调用链之间持续时间的剧烈波动也增加了模型训练收敛的难度。为了解决这个问题，我们将这四类时间相关特征整合为一个向量进行时间信息提取，并将其投影到高维空间中^[67]。

对于每一个 Span 内的状态码特征，其反映了 HTTP 协议中服务器响应客户端请求的数值结果。这些代码通常分为五类：信息性状态码、成功状态码、重定向状态码、客户端错误状态码及服务器错误状态码。例如，“200”表示请求成功。引入状态码有助于理解请求结果并辅助采取相应措施。在本研究中，我们采用独热编码（One-hot Encoding）方法对状态码进行处理。

从直观上看，调用链展现出复杂的层级调用结构，适合表征为有向无环图（DAG）。在这个调用链抽象图中，每一个节点代表一个 Span，而每一条边代表一对 Span 之间的父子调用关系。而每一个节点的 Span 的表征向量由三个基础特征编码的部分组成，分别是语义特征向量，时间特征向量以及状态码特征向量。

（二）GNN 训练

我们使用图神经网络（GNN）来提取调用链数据的结构和关联关系特征。在训练完图神经网络之后，我们使用它对每一条调用链数据进行编码，得到一个固定长度的向量表征。这种表征方式能够简洁地概括调用链的显著特征，同时保留其固有的结构信息及 Span 间的依赖关系。特别地，我们选择使用图注意力网络（GAT）^[68]进行训练和编码，它是一种基于多头自注意力机制的 GNN 变体。能够以更具自适应性且更强表达能力的方式从图结构数据中学习特征表示。

图对比学习（GCL）是一个典型的图神经网络训练的方法。这种方法通过对

比相似和不相似的图来强调特征表示的学习和优化。图对比学习的主要目标就是激励模型在向量空间中将相似图的表示向彼此靠拢，同时将不相似图的表示分离。

为了使相似的调用链具有相近的向量表示，我们借鉴 He 等人^[69]的思路，利用预处理后的调用链并结合特定领域知识来训练 GNN。模型的输入由三条调用链组成的三元组（记为 A, B, C ）构成，旨在预测它们的相对关系。具体任务是判别调用链 A 与 B 的相似度是否高于其与 C 的相似度。

首先，参照^[69]，我们生成了大量的调用链三元组。我们主要关注不同调用链之间的结构与时间延迟的差异。一方面，我们为每一条调用链生成一个路径 ID，它能够唯一地标识每一个调用路径，以此去判断不同的调用链是否具有相同的结构^[29]。两条调用链具有相同的结构特征意味着每个节点的位置必须是一样的 Span 名称和父子调用关系。另外一方面，调用链时延表示了用户端到端请求的时间间隔。直觉上来说，正常调用链与异常调用链之间存在明显的时间延迟差异。

我们使用图神经网络针对每一条调用链进行编码，为之前生产的训练三元组中的调用链 A 、 B 和 C ，分别生成向量表示 G_A 、 G_B 和 G_C 。通过学习它们之间的相对相似性特征，GNN 训练的目标就是通过对不同调用链之间的相似度特征准确地提取调用链的向量表征。我们使用余弦相似度（Cosine Similarity）来衡量两条调用链之间的距离 $d(\mathbf{x}, \mathbf{y})$ ，如公式（3.1）所示：

$$d(\mathbf{x}, \mathbf{y}) = 1 - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \quad (3.1)$$

我们采用三元组损失（Triplet Loss）^[70]训练模型，旨在最小化 G_A 与 G_B 之间的距离，同时最大化 G_A 与 G_C 之间的距离。损失函数 $L(A, B, C)$ 定义如公式（3.2）所示：

$$L(A, B, C) = \max \{d(G_A, G_B) - d(G_A, G_C) + \text{margin}, 0\} \quad (3.2)$$

其中， margin 是一个固定常数，用于控制两个距离之间的间距。通过反向传播算法最小化三元组损失，GNN 对不同调用链的表征与区分能力将逐步达到理想的状态。

（三）向量化表征

我们把训练好的图神经网络当作特征提取器，每条调用链经过预处理后，会被它映射成一个空间向量。这样一来，调用链里那些复杂的结构和属性信息都被保留下来，相似的调用链在向量空间里就会靠得更近。

具体做法是：把预处理好的调用链图数据输入到训练好的图神经网络里，模型会为每一条调用链输出一个 64 维的向量。这个向量表示了每条调用链的独特特征，从而便于后续的分析和比较任务。

三、异常标注模块

在此阶段，未标注的调用链数据需要经过人工标注，以融入操作人员的经验与判断。核心目标是在保证标注质量的前提下，尽可能减少人工标注的工作量。在这种需求下，主动学习^[56]作为一种契合的机器学习范式，提供了可行的解决思路。其基本思想是：通过引入人工反馈，从大量未标注数据中筛选出信息量最丰富的样本进行标注，从而在控制标注成本的同时，尽可能提升模型性能。因此，问题的关键就在于如何准确衡量哪些调用链数据最值得标注。

将调用链用图神经网络转化为向量表示后，每条调用链都对应了一个特征向量。这样一来，相似的调用链在向量空间里会靠得更近，差异大的则离得远。利用这个特性，我们可以对那些结构相近的调用链进行聚类——同一个类别里的样本，只需挑出最典型的一条人工打上标签，其他成员的标注就可以自动推演出来。具体操作上，我们通过聚类算法对调用链分群，然后选取每个簇的中心点作为代表交给人工确认，这样既保证了标注质量，又大幅减少了重复劳动。

为了让标注过程更灵活，操作员可以根据需要自己决定标注多少条调用链。比如说，如果打算标 k 条，系统会先把数据集聚成 k 个簇，然后从每个簇里挑出最靠中心的那条调用链，交给操作员来标。可以想见，随着标注的样本越来越多，整个标注集的准确率也会逐渐向 100% 靠近——这个趋势我们会在本章第五节具体分析。

不过，随着数据量越来越大，再加上已标注的调用链不断累积，聚类的计算成本和存储压力也越来越大。很多传统的聚类算法在这种规模下运行存在很大的压力和困难^[71]。为了解决这个问题，我们参考了 Huang 等人^[72]的思路，改用了一种混合代表性选择的策略。先从整个数据集里抽取十分之一的样本，保证这批样本在向量空间里的分布跟原始数据大致一致。然后在这批样本上进行

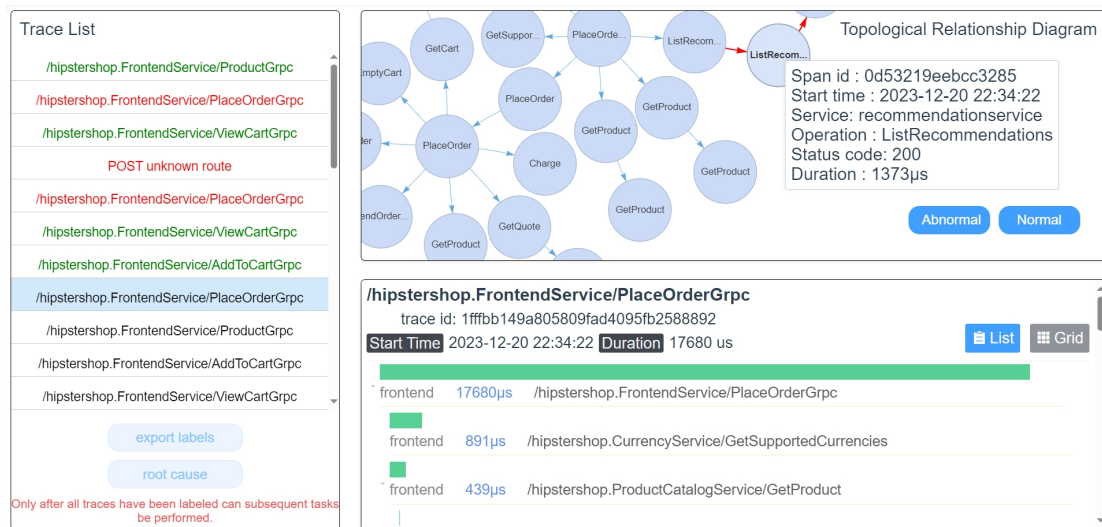


图 3.2 调用链异常标注交互界面

k-means 聚类，聚出 k 个簇，再把每个簇的中心点挑选出来，作为代表数据给操作员标注。这样一来，操作员只需要标这 k 条数据，就能在保证效率的同时，把计算开销也降下来。

我们在 LabelEase 界面上把调用链的拓扑结构直接呈现出来，每个 Span 之间的关系一目了然。这样操作人员就能直接看出整个调用链的调用关系。同时，为了让延迟信息也更清楚，每个 Span 旁边还配了时间轴，能直观看到它在哪个层级、花了多长时间。依赖这些可视化设计，操作人员在标记异常调用链的时候，就能又快又准地做出判断。

第三节 Trace 异常标注工具使用 and 介绍

我们将 LabelEase 方法无缝集成到了定制化的标注工具中，前端使用 Vue，后端使用 Django 框架。图 3.2 展示了调用链异常标注的具体交互界面，该界面以可视化方式整合了多维信息，旨在辅助运维人员快速、准确地完成标注任务。

在 LabelEase 挑出需要标注的代表性调用链之后，操作人员会在图 3.2 这个页面上，对这些选出来的调用链逐一打标签。页面里把跟这条调用链相关的信息都直观地展示出来了，包括结构拓扑长什么样、每个 Span 花了多长时间、状态码多少、操作名叫什么，以及它们在层级里是怎么嵌套的。另外，为了帮操作人员做判断，LabelEase 还把整个数据集里跟这条调用链结构相似的路径的平均耗时、标准差这些统计信息也放在旁边作为参考。这样一来，操作人员在标某条

调用链是否异常的时候，不光能看到它本身的情况，还能知道它在同类里算不算“反常”。通过提供这一全面的标注环境，LabelEase 有效助力运维工程师高效地对异常调用链数据进行标注。

页面左侧采用列表形式展示当前批次待标注的调用链，每条调用链均通过颜色进行状态标记：绿色表示已标为“正常”，红色表示已标为“异常”，黑色表示尚未标注。操作人员在标注时，可以结合右上角的结构信息和右下角的延迟信息来判断，然后点一下对应的按钮，就能给当前调用链打上“正常”或“异常”的标签。另外，运维工程师还可以根据具体情况自定义待标注的调用链数量，这一点将在本章第五节中进一步探讨。整体来说，这套设计让标注流程更灵活。这种灵活的设计使得标注流程能够适配不同规模的数据集与多样的运维需求。

在完成对已提供调用链数据的标注后，LabelEase 能够自动对剩余的未标注调用链数据进行标注，从而减轻人工标注的工作负担。操作人员仅需手动标注少量调用链数据，即可使整个调用链数据集获得全部高质量的标注结果。此外，所有的调用链标注数据均可直接导出，且具有极高的准确率。

最终，运维工程师可以导出 Trace 异常标注的标签文件，格式为标准 CSV 文件。文件中包含两列：第一列为 Trace ID，唯一标识一条调用链；第二列为二分类标签，用‘0’表示正常调用链，‘1’表示异常调用链。该标签文件可直接用于后续的异常检测模型训练、根因分析算法评估或其他智能运维任务，为数据驱动的微服务可观测性研究提供可靠的真值基础。

第四节 数据和对比方法

一、数据集

为了验证 LabelEase 的实际效果，我们在两个数据集上做了实验，分别记为 $\mathcal{D}_1$ 和 $\mathcal{D}_2$ 。这两个数据集里的调用链和对应的故障根因，都经过专业运维人员手工标注，质量上相对有保障。

$\mathcal{D}_1$ 来自我们自己搭的一个微服务系统，底层用的是开源电商应用 Online Boutique^[73]，覆盖了用户浏览商品、加购物车、下单这些典型操作。为了制造异常样本，我们用 Chaos Mesh^[74] 注入了各种故障，比如 Pod 异常、网络延迟、丢包、内存异常等等。最终实验的数据集正常调用链有 103078 条，异常的 27285 条。

②2 来自某家大型商业银行的微服务系统，运行的是真实业务，包含账户管理、账单支付、转账、理财等业务功能。该系统规模大、链路长，数据也更复杂一些——不仅有调用链，还拥有全链路日志和时序指标。我们从里面抽取了 112786 条调用链来用，其中异常调用链有 19207 条，主要用于评估 LabelEase 在这种真实生产环境下的表现。

二、对比方法介绍

为了全面评估 LabelEase 在异常标注任务上的性能，我们选取了近年来提出的六种具有代表性的调用链无监督异常检测方法作为基准。这些方法涵盖了不同的技术路径与建模思路，能够较为全面地反映当前该领域的研究进展。具体包括 MultimodalTrace^[24]、TraceAnomaly^[12]、CRISP^[13]、TraceCRL^[14]、TraceVAE^[15]以及 TraceSieve^[6]。上述方法在各自的原始研究中均表现出优于传统基线模型的检测性能，并在学术界获得广泛关注与引用，因此作为本文的对比基准具有较好的代表性。

这些无监督算法直接利用给定的调用链数据进行训练与标注，以适配我们的调用链标注应用场景。通过将这些方法与 LabelEase 进行对比，我们旨在评估：在训练数据中存在大量异常调用链的现实情况下，这些最先进的异常检测模型是否仍能具备稳健的数据集标注能力。换言之，我们关注的是这些方法在现实工业环境中的鲁棒性——面对数据噪声、异常比例波动以及故障模式多样性，它们能否有效区分正常与异常行为，从而为标注任务提供可靠的候选集。通过这一对比，我们希望揭示无监督方法在辅助标注任务中的潜力与局限性，也为 LabelEase 的改进方向提供实证依据。

三、实验环境与超参数

本实验在一台配备高性能计算资源的服务器上完成，以满足图神经网络训练与大规模调用链数据处理的需求。该服务器配置了两颗 Intel(R) Xeon(R) Gold 5218 CPU，每颗 CPU 拥有 16 个物理核心和 32 个逻辑线程，主频为 2.30 GHz，能够有效支持数据预处理、特征提取等并行计算任务。图形计算方面，服务器搭载了一块 NVIDIA(R) Tesla(R) V100S GPU，显存为 32 GB，具备张量核心与混合精度训练能力，用于加速图神经网络的前向传播与反向求导过程。系统内存为 192 GB，确保在加载大规模调用链数据集时无需频繁进行磁盘 I/O 交换，从

而提升训练效率与稳定性。

LabelEase 框架采用 Python 3.7.0 作为主要编程语言，并基于 PyTorch 1.5.0 深度学习库进行模型构建与训练。PyTorch 提供的动态计算图机制与自动求导功能，为图注意力网络的实现提供了较高的灵活性与开发效率。在图神经网络训练阶段，我们将批大小设为 32，这一取值在显存占用与梯度估计稳定性之间取得了平衡，既避免了因批度过大导致的显存溢出，又确保每次参数更新时具有足够的数据多样性。学习率初始化为 0.0001，采用 Adam 优化器进行参数更新，该学习率经过多轮调试验证，能够在训练初期稳定收敛而不发生震荡。模型总共迭代训练 15 轮，每轮完成后在验证集上计算损失值与评估指标，并保存最优参数用于后续标注任务。上述超参数配置在实验过程中保持一致，以确保不同方法之间的对比具有公平性与可复现性。

四、评价指标

在标注过程中，LabelEase 提供了信息丰富且直观的标注界面，以辅助操作人员高效、准确地进行标注决策。操作人员凭借其对系统运行逻辑与异常表现的专业理解，在异常判定和潜在根因识别中发挥着核心作用。理想情况下，调用链数据集的标注目标是在人工参与下逐步逼近 100% 的准确率。基于此，本文在后续评估中假定操作人员标注的异常样本均为正确标注，并以此为基础考察 LabelEase 对整体数据集的标注有效性。

调用链异常标注任务可以被视为针对每一条调用链的二分类问题，即判断一条调用链是否为异常。为了评估模型在该任务中的性能，本文采用广泛使用的评价指标 F_1 -score。在介绍 F_1 -score 之前，首先对其相关的两个基础指标——精确率（Precision）和召回率（Recall）进行说明。

精确率用于衡量模型预测为异常的样本中，实际为异常样本的比例。该指标反映了模型预测结果的准确性，即在所有被判定为异常的调用链中，有多少比例是真正的异常调用链。其计算公式如下：

$$Precision = \frac{TP}{TP + FP} \quad (3.3)$$

其中， TP （True Positive）表示被正确预测为异常的样本数量， FP （False Positive）表示被错误预测为异常的正常样本数量。

召回率用于衡量所有真实异常样本中，被模型正确识别出来的比例。该指

标反映了模型对异常样本的检测能力，即在所有真实异常调用链中，有多少比例被成功检测出来。其计算公式如下：

$$Recall = \frac{TP}{TP + FN} \quad (3.4)$$

其中， FN (False Negative) 表示未被模型识别出的异常样本数量。

在精确率和召回率的基础上，可以进一步计算 F_1 -score。作为两者的调和平均值， F_1 -score 能够在精确率与召回率之间取得平衡，从而更全面地反映模型在异常检测任务中的表现。其计算公式如下：

$$F_1\text{-score} = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (3.5)$$

由于 F_1 -score 综合了两项指标，适用于对二分类问题进行整体评估，因此常被用作异常检测任务中的核心评价指标之一。

第五节 实验结果及分析

在本节中，我们通过广泛的实验来评估 LabelEase 的性能，旨在回答以下研究问题 (Research Questions, RQs)：

RQ1：整体性能评估——与基准方法相比，LabelEase 在调用链异常检测标注的有效性效率方面表现如何？

RQ2：表征能力分析——与现有的调用链向量化方法相比，LabelEase 基于图的调用链表征机制是否更具优势？

RQ3：采样策略对比——结合主动学习的混合代表性选择 (Hybrid Representative Selection) 策略，其表现是否优于传统的聚类方法？

RQ4：参数敏感性分析——运维人员需要标注的样本数量对系统最终的性能表现有何影响？

一、LabelEase 与基准算法的对比分析 (RQ1)

LabelEase 和所有基线方法在调用链异常标注任务上的评估结果如表 3.1 所示。LabelEase 在两个评测数据集上边都表现出了比所有基线方法都要优越的性能，分别达到 0.99 和 0.98 的 F_1 scores。值得注意的是，这样的性能结果实在运维工程师在数据集 $\mathcal{D}_1$ 只标注了 35 条调用链数据和在数据集 $\mathcal{D}_2$ 只标注了 67 条

调用链数据就实现的，显著降低了标注开销。

在所有的基线方法中，MultimodalTrace 的表现是最差的。MultimodalTrace 使用调用链的操作序列训练了一个基于 LSTM 的模型，主要利用记录的时间戳进行重构。然而，它的特征提取过程是相对来说是简单并具有一些局限性的，导致无法在正常数据与异常数据之间建立充分的区分度。

TraceAnomaly 和 CRISP 使用服务调用链向量（STV）表示调用链数据的特征，并通过变分自编码器的重构误差来进行异常检测。然而，这些方法仅整合了调用链中的服务序列和响应时间，忽略了状态码信息。由于 $\mathcal{D}_1$ 的正常数据较多且服务调用逻辑较 $\mathcal{D}_2$ 更为简单，TraceAnomaly 和 CRISP 在 $\mathcal{D}_1$ 上能学习到更准确的正常模式特征，因此其在 $\mathcal{D}_1$ 上的精确率高于 $\mathcal{D}_2$ 。数据集复杂性的差异解释了这两者在不同数据集 $\mathcal{D}_1$ 和 $\mathcal{D}_2$ 上的性能波动。

TraceCRL 对每一条调用链数据构建了一个操作调用图，并且使用对比学习的方法去训练了一个基于图神经网络的模型。然而，尽管 TraceCRL 通过图数据结构和数据增强的策略学习到了很多调用链的特征，但是特征选择对于异常检测仍然是非常重要的。如果模型难以补货所有和异常模式相关的特征，那么它可能会漏报或错误判断异常调用链。

TraceVAE 使用了一个双变量图 VAE 架构和创新的调度层，分别对调用链结构和节点耗时进行编码。虽然 TraceVAE 提出了减小熵差的技术，但它的有效性高度依赖于特定的数据分布和异常类型。因此，在不同的应用场景下，该方法可能需要进一步的适配或优化。

TraceSieve 将变分图自动编码器（VGAE）与弹性权重固化（EWC）相结合，构建了一种无监督异常检测方法。它在对抗训练框架下利用自动编码器架构来过滤噪声数据。因此，TraceSieve 在识别异常方面更为准确，在 $\mathcal{D}_1$ 上表现出极高的精确率并显著减少了误报。遗憾的是，噪声过滤的敏感性直接影响了模型

表 3.1 LabelEase 与各基准方法在两个数据集上的性能对比

方法名称	$\mathcal{D}_1$				$\mathcal{D}_2$			
	Precision	Recall	F_1 -score	Time	Precision	Recall	F_1 -score	Time
LabelEase	1	0.98	0.99	6.53s	0.96	0.99	0.98	21.68s
MultimodalTrace ^[24]	0.2	0.15	0.17	1.9min	0.17	0.15	0.16	1.6min
TraceAnomaly ^[12]	0.94	0.67	0.78	27.2min	0.21	0.2	0.2	22.2min
CRISP ^[13]	0.8	0.57	0.67	26.1min	0.24	0.21	0.23	17.3min
TraceCRL ^[14]	0.39	0.28	0.33	8.9h	0.48	0.43	0.45	7.2h
TraceVAE ^[15]	0.4	0.3	0.35	1.7h	0.14	0.77	0.23	51.3min
TraceSieve ^[6]	1	0.74	0.85	7.6min	0.17	0.15	0.15	9.8min

学习到的特征和模式。过度的敏感性可能导致模型忽略关键信息，将真实数据误判为噪声，从而无法有效过滤噪声。这削弱了模型的泛化能力和预测性能，导致其在 $\mathcal{D}_2$ 上的表现不尽如人意。

总之，如表 3.1 所展示的结果凸显了现有无监督调用链异常检测方法在数据集标注任务中的局限性。这些基线方法表现不好的主要原因在于训练集中存在大量的异常调用链数据，这阻碍了模型准确学习的正常特征表征。正如前边所强调的一样，在我们的数据集标注场景中，训练集和测试集是一样的，并且数据集中也存在很多的异常调用链数据的占比。因此，这些基线方法都无法准确的区分正常和异常的调用链数据。

在效率评估方面，我们评估了所有基线方法和 LabelEase 在调用链异常标注的时间消耗。如表 3.1 所示，我们展示了利用训练好的基准模型进行测试和标注时产生的时间开销。相比于所有的基线方法，LabelEase 的表现最好，具备最短的时间开销和最高的标注效率。基线方法通常具有复杂的神经网络架构，涉及繁重的前向传播运算，导致比较多的时间成本。相比之下，LabelEase 基于主动学习，使用高效的混合代表性选择策略，通过少量的人工标注就可以获取所有的调用链标签。因此，相比于其他的基线方法，LabelEase 在有效性与效率上都具有更好的表现。

二、基于图的调用链表征效果分析 (RQ2)

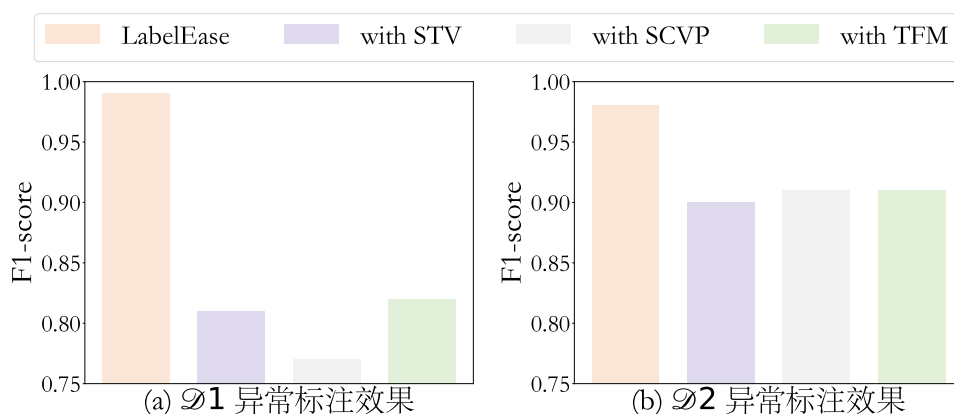


图 3.3 基于图的调用链表征与不同向量化方法在两个数据集上的效果对比

我们在两个数据集上进行了一系列的实验，通过和其他的调用链向量化方法相比，评估了基于图的调用链表征方法的有效性。图 3.3 展示了 LabelEase 使

表 3.2 不同聚类方法对两个数据集的影响

方法名称	$\mathcal{D}1$				$\mathcal{D}2$			
	Precision	Recall	F_1 -score	Time(s)	Precision	Recall	F_1 -score	Time(s)
with K-means	1	0.98	0.99	6.53	0.96	0.99	0.98	21.68
with hierarchical clustering	0.99	0.98	0.99	96.9	0.96	0.95	0.96	51.98
with DBSCAN	0.99	0.92	0.95	116.07	0.83	1	0.91	97.55
with Spectral clustering	0.99	0.98	0.98	200.84	0.96	0.93	0.94	264.24
with random selection	0.8	0.2	0.32	-	0.95	0.33	0.49	-

用基于图的调用链表征方法的效果要优于其他的调用链向量化方法。

在对比实验中,我们将 LabelEase 的图表征替换为服务调用链向量 (STV)^[12]。一个具体的调用链数据的 STV 基于调用路径列表来构建。给定一个调用链数据的 STV 表征中,向量的每个维度的取值代表特定调用路径的响应时间。特别注意的是,如果某一个维度的取值为 -1,则表示该路径不存在。然而,STV 可能没办法捕获复杂系统中普遍存在的深层交互与依赖关系。因此,这种方法很容易忽略 Span 之间关键的调用信息,从而削弱模型的判别性能,甚至导致误报。

随后,我们将图表征替换为服务关键路径向量 (SCVP)^[13]。SCVP 仅对关键路径上的 Span 进行编码。相比于 STV, SCVP 有效降低了特征维度并缩短了推理时间。然而,这种选择性编码策略存在局限性:一方面,仅考虑关键路径可能会遗漏非关键路径上的重要性能干扰信息;另外一方面,关键路径的准确识别高度依赖于领域知识和专家经验,且往往基于对系统结构和运行模式的预设假设。

在采用调用链特征矩阵 (TFM)^[6] 的对比方案中,TFM 提取的特征中,包含调用链的执行时间与等待时间。具体而言,每个 Span 的执行时间记录在 TFM 的第一维,而服务的等待时间记录在第二维。尽管 TFM 引入了额外的时序特征,但它在捕获服务间的调用逻辑以及 Span 间的依赖关联方面仍然存在明显不足。

相比之下,LabelEase 利用训练好的 GNN 为每条调用链生成向量化表征,并且计算在向量空间不同调用链之间的距离。这种基于图的调用链表征方法能够精准捕获微服务系统中调用链复杂的拓扑结构与属性特征。因此,通过该方法获取的调用链信息更加丰富且全面,从而显著提升了 F_1 -score。

三、聚类策略的有效性分析 (RQ3)

我们采用了混合代表性选择 (Hybrid Representative Selection) 策略,为主动学习筛选最具代表性的调用链数据。具体而言,在借鉴 Huang 等人^[72]的思路对调用链集合进行初步采样过滤后,利用聚类方法识别各簇的质心,并将其作为

代表性样本供运维人员进行人工标注。为了确定该任务中最有效且最高效的方法，我们的实验评估涵盖了四种经典的聚类算法。

- **K-means** 是一种迭代聚类算法，通过最小化数据点到其对应簇质心的平方距离之和，将数据划分为 k 个簇。

- **层次聚类 (Hierarchical Clustering)** 通过评估对象间的相似性来构建层级化的聚类结构。

- **DBSCAN** 是一种基于密度的聚类方法，它根据指定半径内包含的最少点数，将物理空间中分布紧密的点划分为簇。

- **谱聚类 (Spectral Clustering)** 通过分析由原始数据派生的相似性矩阵的特征向量 (Eigenvectors)，实现对数据的降维与聚类。

表 3.2 展示了 LabelEase 采用不同聚类算法时的性能与效率表现。实验结果表明，对比过大多数的聚类算法在这个任务中都表现得令人满意。值得注意的是，K-means 算法在所有对比算法中展现出了最好的标注效果和最短的计算时间，证明了它在实际应用中极高的运行效率。因此，我们选择 K-means 聚类算法作为 LabelEase 首选的聚类算法。

此外，我们引入“随机选择 (Random Selection)”作为基线方法，用来对比验证混合代表性选择策略的有效性。从表 3.2 可以看到，随机采样的效果是最差的。由于它不涉及计算开销，我们也没有统计其选择耗时。一方面是之前步骤的基于图的调用链表征让相似的调用链在向量空间里的距离比较近，所以随机采样的精确率的结果还可以；但另一方面，数据集中正常的调用链占了绝大多数，随机采样很容易抽到正常样本，而那些关键的异常样本反倒被漏掉了，结果就是召回率大幅的下降。因此，我们可以看到采用随机选择代替聚类策略无法达到理想的效果，这进一步强调了精准筛选代表性数据点在异常标注任务中的重要性。

四、待标注样本数量的敏感性分析 (RQ4)

我们重点探讨了待标注样本数量的敏感性对 LabelEase 整体标注结果的影响。图 3.4 展示了 LabelEase 的 F_1 -score 随标注样本数变化的趋势。在 LabelEase 中，我们利用 K-means 算法将数据划分为 k 个簇，并选取每个簇中心的调用链进行人工标注。这种设计允许运维人员自主决定标注样本的数量，显著增强了系统的操作灵活性与用户的主观能动性。

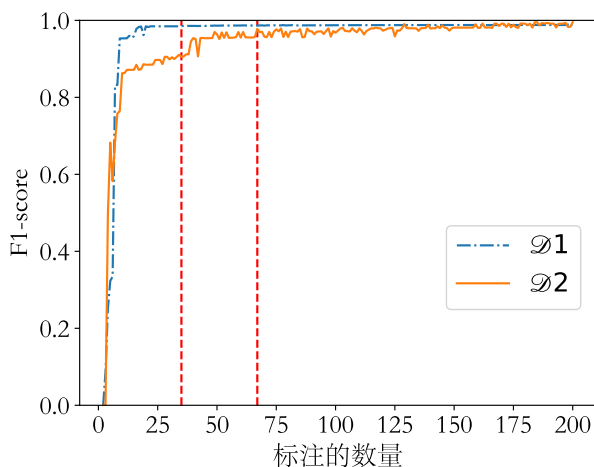


图 3.4 待标注样本数量对标注效果的影响

实验结果显示，随着标注样本数量的增加，LabelEase 的 F_1 -score 稳步提升。然而，更多的标注量也意味着更高的时间成本与人工开销。因此，为了在标注工作量与模型性能之间取得最优平衡，参考图 3.4 中的红色虚线，我们分别选择 35 条和 67 条作为 $\mathcal{D}1$ 和 $\mathcal{D}2$ 的标注样本数。在此设置下，两个数据集上的 F_1 分数分别达到了 0.99 和 0.98。

如果对性能有更高要求，可以进一步增加待标注样本的数量。如图图 3.4 所示，当标注数量达到一定阈值后， F_1 -score 增长放缓并逐渐收敛于 1。我们的实验评估结果表明，对于任何数据集，运维人员仅需将总数据量的 0.1% 作为标注阈值。通过采用这一阈值，我们证明了 LabelEase 的普适性：在将运维人员的标注负担降低 99.9% 以上的同时，依然能够获得高质量的标注数据集。

第六节 本章小节

本章围绕微服务调用链标注中面临的数据规模庞大、拓扑结构复杂等实际困难，介绍了半自动化标注框架 LabelEase 的设计思路与具体实现。该框架首先构建了一个基于图注意力网络的调用链表征模块，通过图对比学习将调用链的拓扑结构、时序信息与状态码等多维信息编码为高维稠密向量。在此基础上，结合主动学习与混合代表性选择策略，利用 K-means 聚类从海量未标注数据中筛选出信息量最大的样本交由专家标注，并进一步借助主动学习机制实现对剩余数据的自动化标注。在实验部分，LabelEase 在基准数据集 $\mathcal{D}1$ 和生产环境数据集 $\mathcal{D}2$ 上进行了验证， F_1 分数分别达到 0.99 和 0.98，明显优于现有的无监督方

法。进一步的消融实验和参数分析表明：基于图网络的表示方法优于传统的向量化手段；运维人员仅需标注约 0.1% 的样本，模型性能即可基本收敛。也就是说，LabelEase 在削减 99.9% 以上人工标注成本的同时，仍能保证生成高质量的标签数据。从更宏观的视角看，这项工作试图回应微服务运维中长期存在的一个痛点——高质量数据集的缺失。通过提供一套可行的半自动化标注方案，也为后续涉及根因分析等需要监督信号的研究打下了一定基础。

第四章 基于 Trace 的微服务系统根因标注

在完成调用链的异常标注后，运维体系面临的下一个核心挑战是如何在已识别的异常调用链中精准定位故障的根本原因（Root Cause）。虽然第三章提出的 LabelEase 框架通过图表征与主动学习解决了“是否存在异常”的判定问题，但在复杂的微服务架构下，仅仅标注异常状态是不够的，还需要对微服务系统的根因进行标注，以辅助先进根因定位算法进行评估和优化。因此，我们在第三章提出的 LabelEase 的基础上补充了根因标注的特性。我们提供一个可视化的前端页面，结合调用链异常区间聚合并辅之于谱分析方法，辅助运维工程师进行微服务系统根本原因的定位标注，提高标注效率和减少标注成本。

第一节 引言

在借助 LabelEase 完成异常调用链的高效标注之后，运维诊断的下一个核心问题随之浮现：这些异常究竟由何种原因引发的？换言之，系统可观测性的关注点需要从“是否发生异常”的状态判别，转向“异常何以发生”的因果溯源。然而，在实际工业场景中，即便异常已被准确识别，要完成对故障根因的定位与标注，仍面临多方面的结构性困难。这一困境首先源于调用链内部依赖关系的复杂性——以典型的电商系统为例，一次下单请求可能涉及认证、库存、支付、物流等十余个微服务，各服务之间既存在串行调用，也包含并行分支与异步回调逻辑。异常可能在数据库访问层悄然萌生，却在向上传播的过程中被多次重试机制或超时设置所掩盖，最终在用户界面表现为“下单失败”的模糊错误。更为棘手的是，根因线索往往并非集中于某一条孤立的调用链，而是分散于时间维度的多条调用链样本之中。例如，某次缓存服务性能抖动，可能在最初几秒仅影响少数请求，随后逐渐扩散为大规模超时——这一演化过程的完整因果链条，远非单条调用链所能呈现。

通常情况下，根因标注难以依托于单一异常调用链完成，而需要综合考察一段时期内产生的多条调用链样本。以故障发生前后 30 秒为观察窗口，运维人员往往需要横向对比数十万乃至百万量级的调用链数据，才能从统计意义上识别出异常模式的变化趋势。这类跨样本的联合分析对于还原故障演化过程至关重要，例如，通过对比故障时刻与正常基线下的调用链分布，可以发现某服务的平

均耗时从 50 毫秒陡增至 2 秒，或某条调用路径的出现频率从 0.1% 激增至 30%。然而，这种时间窗口内的多样本分析，也使得数据处理维度从单条调用链的图结构复杂度，扩展至跨样本的分布演化复杂度，呈几何倍数增加了分析的难度。

运维人员在确定根因时，需系统梳理该时间段内所有相关异常调用链，并从中进行模式识别与归纳，以得出最终的根因结论。在实际操作中，这意味着工程师需要依次检查数百个异常服务实例、数千条调用路径的耗时波动、以及数万条调用链的拓扑变化——这些信息散落在不同的监控面板、日志文件和追踪系统中，彼此之间缺乏关联索引。面对微服务系统产生的高频、海量且具有级联效应的异常数据，人工逐条审查不仅难以具备可操作性，还极易因信息过载导致误判。例如，在出现网络分区故障时，大量服务会同时上报连接超时，若缺乏系统的归纳能力，标注者很容易将表象上的大规模异常误判为多个独立故障，而忽略其背后单一的物理网络根源。

具体而言，由于服务间存在复杂的调用依赖，底层性能瓶颈往往触发一系列连锁反应，致使大量上游服务在观测界面中呈现异常状态。以某次数据库连接池耗尽为例，数据库响应变慢会立即影响到所有依赖该库的服务，这些服务的延迟增加又会传递给它们的调用方，形成一条从底层到顶层的“异常瀑布”。在此场景下，若缺乏有效的辅助工具，标注者很难从数十个同时表现为异常的服务节点中，剔除由于级联效应产生的干扰信息，精准锁定真正的故障始作俑者——即那个最先出现性能劣化的数据库实例。传统的监控告警往往在同一时刻触发成百上千条警报，将真正的根因淹没在噪声之中，这正是当前可观测性工具链在根因分析维度上的典型短板。

针对上述难题，本章在 LabelEase 异常标注能力的基础上，进一步构建了面向根因的辅助标注模块。该模块通过自动聚合异常时间窗口内的多维 Trace 数据，引入基于谱分析的故障定位技术，从海量调用链中自动识别出故障传播路径，并依据各服务在正常与异常样本中的行为差异，计算其异常嫌疑度。以一次典型的微服务故障为例，该模块能够将数万条调用链、上百个服务节点的原始信息，压缩为一张根因候选拓扑和一份按异常分数排序的服务列表，将运维专家需要审视的数据规模削减数个量级。这一设计旨在通过自动化手段降低专家的认知负荷，实现从异常发现到根因确认的完整闭环，为后续基于监督学习的根因分析研究提供高质量的真值数据支撑。

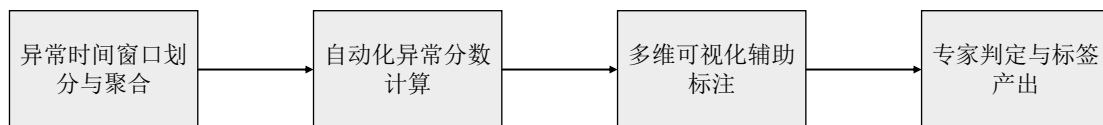


图 4.1 LabelEase 根因标注流程图

第二节 方法介绍

一、整体流程概述

本节对基于 Trace 的微服务系统根因标注流程进行系统性综述。根因标注是在完成调用链异常识别后的深层分析阶段，旨在从时间相关的一组异常样本中锁定故障源头。LabelEase 的根因标注流程主要分为异常时间窗口划分与聚合、自动化异常分数计算、多维可视化辅助标注、专家判定与标签产出四个阶段，如图 4.1 所示。其核心逻辑架构旨在解决多变量干扰下的根因判定难题，具体执行流程如下：

1. 异常时间窗口划分与聚合：系统根据第三章中异常标注模块产生的实时标签，自动识别异常波发的起止时刻，并以此划分异常时间段。位于同一时间窗口内的所有异常调用链会被提取并聚合为一个分析集合，作为后续根因分析的输入样本。

2. 自动化异常分数计算：针对聚合后的样本集，系统采用谱分析故障定位（SBFL）技术。通过统计服务节点在正常与异常调用链中的覆盖频率计算各服务的异常得分。该步骤将海量的原始数据压缩为具有优先级的待核查服务清单。

3. 多维可视化辅助标注：为弥补统计学分析对拓扑依赖和时延细节忽视的不足，LabelEase 在前端通过颜色深度直观展示各服务的异常程度，并同步呈现服务间的调用依赖拓扑与时延分布。

4. 专家判定与标签产出：运维专家结合自动化提供的异常程度排名、拓扑传播路径及响应时间异常，利用专业经验对故障根因进行最终确认。标注结果随后作为高质量根因标签导出，用于构建生产级的故障定位数据集。

通过上述流程，LabelEase 实现了从“孤立的异常判定”到“结构化的根因溯源”的平滑过渡，显著提高了微服务系统根因标注的效率。

二、异常数据聚合与时间窗口划分

根因标注的首要步骤是建立异常事件与时间维度的关联，即将离散的异常调用链样本按照其发生时刻组织为可供分析的时间单元。根据第三章得到的调

用链异常标注结果，系统自动识别异常爆发的起止时间，并据此将连续的时间轴划分为若干个异常时间段。这一划分的核心依据在于：根因判定本质上是一个从“现象群”中提炼“核心源”的过程，单一调用链的特征往往不足以消除随机网络抖动或瞬时毛刺带来的噪声干扰。例如，某次数据库连接池耗尽故障可能表现为持续 30 秒的性能劣化，期间产生数万条异常调用链，但其中任意一条单独观测时，可能仅显示某次查询超时，难以揭示其背后是连接资源耗尽这一系统性原因。因此，根因判定通常需要考察一段时间内多条调用链的共性特征，通过统计规律从个体现象中剥离出本质因素。为此，LabelEase 对处于同一异常时间段内的所有调用链进行空间维度的聚合，将时间上连续、空间上分散的异常样本组织为一个统一的故障分析单元。

这种时间窗口的切分方式，既为后续的数据分析和人工排查提供了相对完整且具有统计意义的样本基础，也通过聚合操作将海量的原始调用链数据压缩为若干信息密度较高、数量可控的“故障快照”。以某电商系统的大促场景为例，系统每分钟产生的调用链数量可达千万级，若直接对所有原始数据进行全局分析，无论是计算开销还是人工审查的可行性都难以承受。通过异常时间窗口的划分，系统可将连续的观测数据划分为若干个异常区块，每个区块对应一次独立的故障事件或性能抖动。这一过程不仅大幅降低了后续分析的数据规模，也为跨样本的模式识别提供了结构化的输入——例如，在某一时间窗口内，系统可以统计各服务的平均耗时、错误率变化趋势，以及调用路径的分布偏移，这些统计量远比原始调用链更适合作为根因分析的输入特征。在大规模分布式系统中，根因定位所面临的计算效率问题和时序对齐问题，很大程度上正是依靠这一聚合步骤来实现初步的降维与规整，为后续的精细化分析奠定基础。

三、基于谱系故障定位的微服务异常分数计算

在微服务架构中，故障往往并非孤立出现，而是通过复杂的调用依赖关系在系统中扩散。一个常见的观测现象是，若某服务在大量异常调用链中被频繁访问，而在正常调用链中极少出现，则该服务自身存在故障的可能性较高^[5,28]。这一直觉与软件工程中的谱系故障定位思想高度契合——通过比较程序元素在成功与失败执行中的出现频率，可以推断其可疑程度。本研究将这一思想引入微服务调用链分析，通过统计各服务在已标注正常与异常调用链中的覆盖情况，定量评估每个服务的故障嫌疑。

为实现上述分析，本研究采用了谱系故障定位（Spectrum-based Fault Localization, SBFL）技术。针对服务集合 P 中的任意服务 $O \in P$ ，SBFL 首先统计以下四类关键指标，用以刻画该服务在调用链数据集中的分布特征：

- O_{ef} ：覆盖该服务且被标注为异常的调用链数量。该指标反映了服务与故障现象之间的正相关性。

- O_{ep} ：覆盖该服务但被标注为正常的调用链数量。该指标用于衡量服务在正常情况下的活跃程度，有助于排除误报。

- O_{nf} ：未覆盖该服务却被标注为异常的调用链数量。该指标代表了不由当前服务引发的其他故障样本。

- O_{np} ：未覆盖该服务且被标注为正常的调用链数量。该指标提供了全局正常样本的基准。

上述统计量可通过 LabelEase 标注后的数据集直接计算获得，它们共同构成了谱分析的基础。在软件工程领域，SBFL 已被广泛证明在程序故障定位中具有良好的稳健性和可解释性^[26]。将其迁移至微服务调用链场景，其核心思想保持一致：将服务视为程序中的“语句”，将调用链视为“测试用例”，通过对比服务在正常与异常执行中的出现频率差异，推断其故障嫌疑。基于上述统计量，我们采用经典的 Ochiai 系数计算服务 O 的异常分数 $Score_O$ ，其公式如下：

$$Score_O = \frac{O_{ef}}{\sqrt{(O_{ef} + O_{ep}) \cdot (O_{ef} + O_{nf})}} \quad (4.1)$$

该公式在诸多谱分析研究中表现出对故障元素较高的区分能力，能够平衡服务的异常覆盖率和整体异常样本比例，有效抑制因服务调用频率本身较高而造成的虚警。例如，在某次数据库连接池耗尽故障中，数据库服务 O 的 O_{ef} 极高而 O_{ep} 极低，公式将赋予其高分；而受级联影响的上游服务虽然 O_{ef} 也可能较高，但其 O_{ep} 往往同样不低，因而得分相对较低，从而实现从噪声中凸显真实根因。

计算完成后，系统根据异常得分对服务节点进行可视化着色处理，利用颜色的深浅直观地向运维人员展示不同服务的风险程度，从而辅助其迅速锁定潜在的故障源。

四、多维特征融合与专家辅助决策

尽管基于谱系的分析方法在统计学上具有一定的指导意义，但其固有的局限性同样不容忽视。这类方法主要依赖调用链中服务的频率分布作为核心判别依据，却忽略了微服务系统中同样关键的时延特性与拓扑依赖关系。以一次典型的数据库连接池耗尽故障为例，当底层数据库服务响应变慢时，所有上游调用服务（如订单服务、库存服务）的耗时均会显著增加，导致这些服务在异常调用链中的出现频率大幅上升。单纯依靠频率统计，可能会将多个上游服务均列为高嫌疑对象，而真正的根因——那个最先出现性能劣化的数据库实例——反而可能被淹没在级联效应的噪声之中。类似地，在服务间存在深度耦合或强依赖的复杂场景下，若某一基础服务发生短暂抖动，其上游服务往往会呈现“多点爆发”的异常表象，频率统计难以从统计学层面区分因果与相关，从而影响根因定位的准确性^[26]。

为此，LabelEase 在前端界面中集成了服务调用拓扑与时延信息的综合可视化模块，旨在通过多维信息的融合辅助专家进行精准判定。具体而言，该模块的设计从以下几个方面增强标注的准确性：

- 拓扑感知：展示服务间的父子调用关系，帮助运维人员识别故障的传播路径及其引发的级联效应。
- 时延洞察：结合各 Span 的执行耗时，辅助专家辨析是否存在由于性能瓶颈导致的非报错型异常。
- 专业知识介入：通过直观的多维信息呈现，充分发挥运维专家的领域经验，在自动化建议的基础上进行最终的根因判定。

综上所述，通过将自动化的谱分析技术与交互式的多维可视化相结合，本方法实现了从海量异常数据到精准根因标签的高效转化。

第三节 微服务系统根因标注工具使用 and 介绍

在完成调用链数据的异常标注后，运维工程师将进入微服务系统故障根因标注界面，如图 4.2 所示。LabelEase 根据先前标注的异常调用链的时间戳信息，自动识别故障发生的时间区间，并将连续时间轴上的异常事件合并为若干个独立的故障时间段。这种合并机制以异常爆发密度和持续时长为依据，确保每个时间段内聚合的调用链在时间上连续、在逻辑上相关。例如，若某次数据库微服

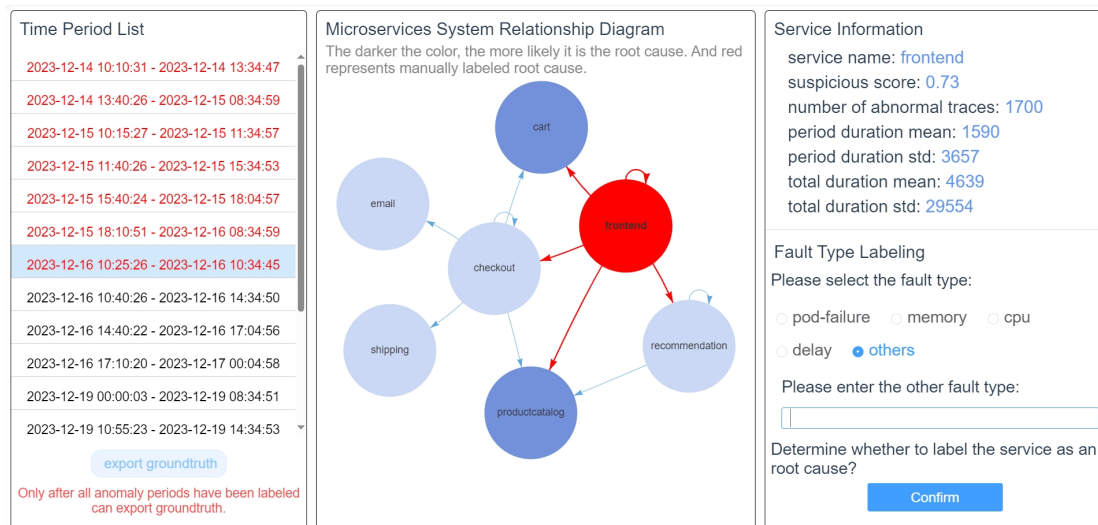


图 4.2 微服务系统根因标注交互界面

务节点故障导致持续 30 分钟的性能劣化，期间产生的数万条异常调用链将被归入同一个故障时间段，从而为后续分析提供统一的样本集合。在每个故障时间段内，系统对调用链进行聚合，并基于谱分析故障定位技术为每个微服务计算异常分数。该分数反映了服务在异常调用链中的出现频率相对于正常基线的高低，为工程师判定根因提供量化参考——分数越高的服务，越有可能是故障的源头。

然而，谱分析方法仅考虑服务在调用链中的出现频次，却忽略了微服务系统中同样关键的时延特性以及服务间的拓扑依赖关系。当多个具有紧密耦合关系的服务同时卷入异常请求时（例如，上游服务因下游延迟而被波及），仅依赖频率统计可能难以区分真实的故障源与被动受影响的服务。为弥补这一局限性，LabelEase 在标注界面中整合了多维辅助信息：包括每个微服务节点在故障时间段内的平均处理时延，以及故障期间服务调用关系的动态拓扑图。此外，系统还提供整个数据集的全局统计信息，如所有调用链经过各服务节点的时延平均值与标准差，作为衡量当前行为是否偏离正常的基准。例如，若某服务节点的平均时延从全局基准的 50 毫秒飙升至故障期间的 2 秒，即使其异常分数并非最高，也可能成为优先关注的候选。凭借这些丰富的上下文信息，运维工程师可以结合自身的领域经验，精准识别引发故障的根本原因，避免被级联效应产生的虚假表象所误导。

对于每一个故障时间段，一个微服务系统的调用关系图会被展示在工具标注页面的正中央。图中每个服务节点根据计算所得的异常分数被赋予不同的颜

表 4.1 标注真值数据集字段语义定义

字段名称	数据类型	语义描述
service	List[String]	根因服务标识 : 引发系统故障的原始微服务组件或实例 ID。
failure_type	List[String]	故障类型标签 : 故障的物理属性或逻辑分类。
timestamp	List[Integer]	发生时间戳 : 故障注入或被观测到的开始时间戳。
duration	List[Integer]	影响持续时间 : 表示故障发生的时间。

色深度: 分数越高, 节点颜色越深, 表示其作为根因的嫌疑程度越高; 分数越低则颜色越浅, 表示其受级联影响的可能性更大。例如, 在某次订单服务超时故障中, 数据库节点因其异常分数最高而显示为深色, 而上游的订单服务和支付服务则呈现浅色, 运维工程师可以据此优先排查数据库实例。一旦选中一个微服务作为候选, 那么一些更细节的信息将会被展示到右边的框里。运维工程师可根据这些信息选择具体的故障类型 (如 “CPU” 或 “网络” 等), 并完成对该故障时间段根因的标注。如果真实的根本原因故障类型没有包含在既定的选项里边, 运维工程师也可以选择 “其他” 选项, 并在输入框输入真正的故障类型被标记为根因的服务节点在拓扑图中将高亮为红色, 便于后续复核与追溯。

完成所有故障时间段的根因标注后, 系统可将标注结果与原始调用链数据一同导出, 形成包含完整真值标签 Ground Truth 的高质量 Trace 数据集。该数据集不仅记录了每条调用链的异常属性, 还标定了故障发生的时间窗口及其对应的根因服务, 为后续基于监督学习的异常检测、根因定位等研究任务提供了坚实的数据基础。例如, 研究者可基于此数据集训练深度神经网络模型, 使其自动从调用链中识别故障模式, 或将根因定位问题转化为图节点分类任务。最终, LabelEase 通过从异常检测到根因标注的完整闭环, 为微服务可观测性研究注入了高质量的数据驱动力。

特别地, LabelEase 最后会把标注好的 Ground Truth 统一导出成标准的 JSON 格式, 方便后续复用和研究。因此, 专家的经验就被转化成了结构化的元数据, 不管是训练异常检测模型, 还是评估根因定位算法, 都有了统一的数据基准。故障根因的 JSON 格式文件的包含四个字段: “service”、“failure_type”、“timestamp”、“duration”, 各字段的含义在表 4.1 里做了详细说明。

第四节 实验结果及分析

在本节中，我们通过实验来评估 LabelEase 在微服务系统根因标注的准确率。针对实验数据集和实验环境，我们仍然实验第三章第四节的配置。

在完成所有异常 Trace 的标注后，我们进一步评估所提出方法在根因定位阶段的有效性。具体而言，我们利用 LabelEase 对异常 Trace 进行聚合，并根据异常 Trace 的时间分布自动识别故障发生的时间区间。由于根因标注过程中引入了运维人员的专业经验与人工判断，我们假设运维人员能够为每一个故障时间区间正确标注其真实根因。因此，在这一阶段的评估重点在于：系统是否能够准确识别数据集中所有故障发生的时间区间，从而为后续的根本标注提供可靠基础。

由于数据集 $\mathcal{D}2$ 中包含的故障案例数量较少，不足以支撑统计显著的定量分析，本实验仅在 $\mathcal{D}1$ 数据集上进行评估。我们将 LabelEase 自动聚合得到的故障时间区间与数据集中提供的人工标注真值进行逐点对比，采用逐点 PA (Point-wise Agreement) 指标衡量检测准确性。PA 指标计算每个时间点上系统输出与真值的匹配程度，适用于评价故障区间检测任务。但需注意，当某些异常时间段持续时间较长时，PA 指标可能因多数时间点被正确覆盖而产生分数膨胀现象^[75]。实验结果表明，LabelEase 在故障时间区间检测任务上的 Precision 达到 0.89。这意味着，在所有被系统识别为“故障区间”的时间段中，有 89% 确实对应真实故障。这一结果说明，只要异常能够在调用链层面留下痕迹，LabelEase 通常能够较为准确且高效地锁定故障发生的时间窗口，从而为专家的根本标注提供可靠的候选范围。

然而，在故障时间区间的识别过程中仍然存在较多的漏检 (False Negative)，其 Recall 仅为 0.44。造成这一现象的原因主要包括以下两个方面。首先，某些故障类型的异常特征并未在 Trace 数据中体现，而是主要反映在多变量时间序列 (MTS) 数据中，例如基础设施层面的指标波动。以 Pod 故障为例，当某服务实例因 OOM (内存溢出) 被 kill 时，该实例无法继续处理请求，导致部分调用链直接中断或缺失，使得 Trace 数据中可能仅出现少数不完整的记录，甚至完全没有异常 Trace 生成。在这种情况下，仅凭调用链分析难以捕捉到故障时间区间。其次，在 $\mathcal{D}1$ 数据集中，未被检测到的故障时间区间主要来源于 Pod 故障 (pod failure)。与之形成对比的是，“loss” (网络丢包)、“delay” (网络延迟)、“CPU” (CPU 过载) 以及 “memory” (内存压力) 等类型的故障通常不会导致服务实例完全不可用，而是以性能退化的形式表现出来。在这类故障中，系统仍能正常记录 Trace

数据，异常往往体现在调用延迟的增加、调用路径的变化或错误码的出现上，因此容易被系统捕捉。值得注意的是，即使是 Pod 故障，当故障根因位于特定服务实例（如 recommendation service 的某个实例）时，由于该实例的不可用可能导致上游服务调用超时或重试，这些异常行为仍可能在 Trace 数据中留下痕迹。因此，我们可以得出结论：调用链能否反映异常，很大程度上取决于故障类型以及故障发生的具体服务实例。例如，数据库连接池耗尽故障几乎必然导致大量调用链超时，而物理机宕机则可能造成观测断崖，使得故障区间难以通过 Trace 数据完整识别。

综上所述，虽然 LabelEase 在 $\mathcal{D}1$ 上的 Recall 指标受到数据源天生特性的限制，但在已捕获的异常区间内，其 0.89 的 Precision 证明了系统在“专家介入前提下的高效性”。这意味着，只要故障在调用链层面留下了可识别的痕迹（如性能退化、错误码激增、路径变更等），LabelEase 就能以极高的置信度将这些异常区间聚合提炼，并直接呈现给运维专家进行根因判定。这一过程将海量的原始调用链数据压缩为极少数高价值的待标注区间，大幅降低了专家筛选数据的认知负担。未来，若能结合多维多源数据（如 MTS 指标、日志信息）进行联合分析，有望进一步提升对各类故障的召回能力，从而构建更全面的故障感知与根因标注系统。

第五节 本章小节

本章在第三章 LabelEase 异常标注框架的基础上，进一步面向微服务架构下的根因标注任务，提出了一套基于 Trace 的辅助标注模块，旨在应对根因定位难度大、故障线索分散、专家认知负担重等问题。首先，本章梳理了根因标注的整体流程，主要包括：异常时间窗口的自动划分与聚合、基于谱分析故障定位技术的异常分数计算，以及融合拓扑结构和时延信息的多维可视化辅助决策。其中，SBFL 方法通过统计各服务在正常与异常调用链中的出现频次 ($O_{ef}, O_{ep}, O_{nf}, O_{np}$)，构建异常评分公式，从而将海量 Trace 数据转化为可供专家参考的候选服务列表。为弥补谱分析方法在时序特征与服务依赖关系方面的信息缺失，本章设计的标注界面同时展示了服务调用拓扑、聚合平均延迟及全局基准信息，帮助运维专家结合领域经验做出更准确的判断。实验结果表明，LabelEase 在故障时间窗口定位任务中表现出较好的可靠性，在数据集 $\mathcal{D}1$ 上的精确率达到 0.89。尽管受限于部分故障类型（如 Pod 故障）的特征表现形式，召回率仅为 0.44，但进一步的

实验分析揭示了 Trace 数据在性能退化类故障上的显著表征优势，以及故障表现与服务实例位置之间的强相关性。总体而言，本章工作实现了从异常发现到根因确认的完整闭环，通过构建高质量的根因标注真值，为后续微服务系统的智能故障诊断研究提供了可复用的数据基础。

第五章 总结与展望

第一节 工作总结

随着云计算和微服务架构的持续演进，现代分布式系统在规模和复杂性方面均呈现出显著增长。在此背景下，调用链数据逐渐成为保障系统可观测性的关键信息来源，基于调用链的异常检测与故障定位也因而成为智能运维研究的重要方向。然而，很多先进的模型和方法在评估和优化时依赖于高质量的标注数据集，而在实际生产环境中，调用链数据不仅规模庞大，结构也较为复杂，依赖人工逐条标注的方式在成本上难以承受，进而制约了相关技术的深入发展与实际部署。因此，如何高效获取高质量的调用链标注数据，已成为当前研究中亟待解决的关键问题之一。

针对这一问题，本文设计并实现了一个半自动调用链标注工具 LabelEase，旨在辅助运维人员高效完成调用链数据的标注任务。该工具通过少量人工标注与自动化模型推断相结合的方式，实现对大规模调用链数据的有效标注，在降低人工成本的同时，力求保证标注质量。围绕该工具的设计与实现，本文开展了系统性的研究工作，主要成果包括以下几个方面：

本文首先梳理了调用链数据在异常检测与根因定位任务中的关键作用，指出当前相关研究在数据标注环节所面临的现实瓶颈。具体而言，结合调用链数据的结构特性与运维场景的实际需求，分析了标注任务的主要挑战，并在此基础上明确了半自动标注工具的设计目标与核心功能需求，为后续系统构建提供了依据和方向。

其次，本文设计并实现了一款半自动调用链标注工具 LabelEase。该工具将模型辅助标注与人工校验相结合，在保证标注质量的前提下有效提升了标注效率。系统支持对调用链数据进行高效管理、微服务根因自动预测以及人工修正，使标注流程更加灵活可控，从而能够在有限人工投入下快速构建高质量的调用链标注数据集。

随后，我们在两个真实数据集上对 LabelEase 进行了实验评估。实验结果显示，仅需少量人工标注的调用链参与，系统即可获得较高的标注准确率， F_1 -score 分别达到 0.99 和 0.98。与完全人工标注相比，该工具可减少至少 99.9% 的标注工作量，显著提升了标注效率。此外，实验还发现，直接采用无监督异常检测模

型对原始数据进行训练并生成标注的方式并不可行，这进一步凸显了本文方法的必要性与实际应用价值。

综上所述，本文提出的半自动调用链标注工具 LabelEase，在有效控制标注质量的同时，大幅降低了人工参与成本，为构建高质量的调用链数据集提供了一条可行路径。该工具不仅为调用链异常检测与根因定位等相关研究提供了可靠的数据支撑，也为智能运维场景下的数据构建与模型训练探索了新的可能性。

第二节 未来展望

尽管本文提出的半自动调用链标注工具在降低人工标注成本和提升标注效率方面取得了良好效果，但仍存在进一步优化和扩展的空间。未来的研究工作可以从以下几个方面展开：

一个可考虑的方向是进一步优化模型训练过程，降低训练时间和计算开销。在当前方法中，模型训练仍存在一定的训练开销，尤其在数据规模较大的场景下更为明显。后续工作可引入迁移学习机制，借助已有系统或数据集上预训练的模型参数，实现新任务下的快速初始化，从而在提升训练效率的同时增强模型的泛化能力。

另外，我们将在 LabelEase 标注工具使用上接入大模型辅助标注。不同微服务系统在调用结构、业务逻辑以及运行负载等方面各不相同，这对标注模型的泛化能力提出了更高要求。如何在系统差异显著的场景下构建更具适应性的标注模型，仍是一个值得进一步关注的问题。基于大模型强大的能力，运维工程师可以将标注的专家经验输入给大模型辅助人工标注，提升标注效率。甚至可以微调一个垂直领域的专家大模型，能够理解微服务系统运维的相关知识，真正成为虚拟运维专家，将经验和数据结合思考和分析，不仅能够标注调用链数据，也能够微服务系统运维其他领域的数据集上充满贡献力。

接下来可以考虑把 LabelEase 进一步往在线服务的方向发展。比如把它部署在服务器上，提供 Web 界面或 API 接口，这样研究人员和运维人员就能通过网络直接使用，标注工作也能变成一种在线的、可多人协作的方式。这样一来，工具用起来会更方便，不同团队之间也更便于共享数据和交流经验。

从长远来看，随着智能运维研究的不断深入，高质量数据集的重要性只会越来越突出。后续可以考虑基于 LabelEase 这套工具，构建规模更大、覆盖场景更丰富的调用链标注数据集，并择机开源出来，供学界和工业界使用。可以预见，

一旦有了更多高质量数据做支撑，基于数据驱动异常检测和故障诊断方法也会跑得更顺，最终也能帮 AIOps 技术往前走一步。

参考文献

- [1] 黄梓程, 陈鹏飞, 余广坝, 等. 面向 Java 微服务系统的透明请求追踪及采样方法[J]. 软件学报, 2023, 34(07): 3167-3187. DOI: 10.13328/j.cnki.jos.006523.
- [2] 罗睿辞, 叶蔚, 刘学洋, 等. 基于拥塞博弈的微服务运行时资源管理方法[J]. 电子学报, 2019, 47(07): 1497-1505.
- [3] 于庆洋, 白晓颖, 李明杰, 等. 基于调用链控制流分析的大型微服务系统性能建模与异常定位[J]. 软件学报, 2022, 33(05): 1849-1864. DOI: 10.13328/j.cnki.jos.006209.
- [4] BI T, PAN Y, JIANG X, et al. VECROsim: A Versatile Metric-oriented Microservice Fault Simulation System (Tools and Artifact Track)[C]//2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE). [S.l. : s.n.], 2022: 297-308.
- [5] LI Z, CHEN J, JIAO R, et al. Practical root cause localization for microservice systems via trace analysis[C]//2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS). [S.l. : s.n.], 2021: 1-10.
- [6] ZHANG S, PAN Z, LIU H, et al. Efficient and Robust Trace Anomaly Detection for Large-Scale Microservice Systems[C]//2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). [S.l. : s.n.], 2023: 69-79.
- [7] LIU D, HE C, PENG X, et al. Microhecl: High-efficient root cause localization in large-scale microservice systems[C]//2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). [S.l. : s.n.], 2021: 338-347.
- [8] SUN Y, SHAO M, NIE X, et al. A Comprehensive Benchmark and Empirical Study of Trace Anomaly Detection[J]. IEEE Transactions on Services Computing, 2025.
- [9] HUANG L, ZHU T. tprof: Performance profiling via structural aggregation and automated analysis of distributed systems traces[C]//Proceedings of the ACM Symposium on Cloud Computing. [S.l. : s.n.], 2021: 76-91.
- [10] ZHANG C, PENG X, SHA C, et al. Deeptralog: Trace-log combined microservice anomaly detection through graph-based deep learning[C]//Proceedings of the 44th International Conference on Software Engineering. [S.l. : s.n.], 2022: 623-634.
- [11] ZENG Z, ZHANG Y, XU Y, et al. Traceark: Towards actionable performance anomaly alerting for online service systems[C]//2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). [S.l. : s.n.], 2023: 258-269.
- [12] LIU P, XU H, OUYANG Q, et al. Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks[C]//2020 IEEE 31st International Symposium

- on Software Reliability Engineering (ISSRE). [S.l. : s.n.], 2020: 48-58.
- [13] ZHANG Z, RAMANATHAN M K, RAJ P, et al. {CRISP}: Critical path analysis of {Large-Scale} microservice architectures[C]//2022 USENIX Annual Technical Conference (USENIX ATC 22). [S.l. : s.n.], 2022: 655-672.
- [14] ZHANG C, PENG X, ZHOU T, et al. TraceCRL: contrastive representation learning for microservice trace analysis[C]//Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. [S.l. : s.n.], 2022: 1221-1232.
- [15] XIE Z, XU H, CHEN W, et al. Unsupervised anomaly detection on microservice traces through graph vae[C]//Proceedings of the ACM Web Conference 2023. [S.l. : s.n.], 2023: 2874-2884.
- [16] ZHOU X, PENG X, XIE T, et al. Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study[J]. IEEE Transactions on Software Engineering, 2018, 47(2): 243-260.
- [17] LI Z, ZHAO N, ZHANG S, et al. Constructing large-scale real-world benchmark datasets for aiops[J]. arXiv preprint arXiv:2208.03938, 2022.
- [18] DENG J, DONG W, SOCHER R, et al. Imagenet: A large-scale hierarchical image database [C]//2009 IEEE conference on computer vision and pattern recognition. [S.l. : s.n.], 2009: 248-255.
- [19] FASEEHA U, SYED H J, SAMAD F, et al. Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms[J]. IEEE Access, 2025.
- [20] Jaegertracing.io. Jaeger[EB/OL]. 2022. <https://www.jaegertracing.io/>.
- [21] Twitter. Zipkin[EB/OL]. 2022. <https://zipkin.io/>.
- [22] SKYWALKING A. SkyWalking[EB/OL]. 2022. <https://skywalking.apache.org/>.
- [23] ZHAO N, ZHU J, LIU R, et al. Label-less: A semi-automatic labelling tool for kpi anomalies [C]//IEEE INFOCOM 2019-IEEE Conference on Computer Communications. [S.l. : s.n.], 2019: 1882-1890.
- [24] NEDELKOSKI S, CARDOSO J, KAO O. Anomaly detection from system tracing data using multimodal deep learning[C]//2019 IEEE 12th International Conference on Cloud Computing (CLOUD). [S.l. : s.n.], 2019: 179-186.
- [25] PEROZZI B, AL-RFOU R, SKIENA S. Deepwalk: Online learning of social representations[C]//Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining. [S.l. : s.n.], 2014: 701-710.

-
- [26] YU G, HUANG Z, CHEN P. TraceRank: Abnormal service localization with dis-aggregated end-to-end tracing data in cloud native systems[J]. *Journal of Software: Evolution and Process*, 2023, 35(10): e2413.
- [27] GAN Y, LIANG M, DEV S, et al. Sage: practical and scalable ML-driven performance debugging in microservices[C]//*Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. [S.l. : s.n.], 2021: 135-151.
- [28] YU G, CHEN P, CHEN H, et al. Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments[C]//*Proceedings of the Web Conference 2021*. [S.l. : s.n.], 2021: 3087-3098.
- [29] GUO X, PENG X, WANG H, et al. Graph-based trace analysis for microservice architecture understanding and problem diagnosis[C]//*Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. [S.l. : s.n.], 2020: 1387-1397.
- [30] ZHANG C, DONG Z, PENG X, et al. Trace-based Multi-Dimensional Root Cause Localization of Performance Issues in Microservice Systems[C]//*Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. [S.l. : s.n.], 2024: 1-12.
- [31] CAI Y, HAN B, LI J, et al. Modelcoder: A fault model based automatic root cause localization framework for microservice systems[C]//*2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*. [S.l. : s.n.], 2021: 1-6.
- [32] SUN Y, SHI B, MAO M, et al. ART: A Unified Unsupervised Framework for Incident Management in Microservice Systems[C]//*2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. [S.l. : s.n.], 2024: 1183-1194.
- [33] DRAGONI N, GIALLORENZO S, LAFUENTE A L, et al. Microservices: yesterday, today, and tomorrow[J]. *Present and ulterior software engineering*, 2017: 195-216.
- [34] JAMSHIDI P, PAHL C, MENDONÇA N C, et al. Microservices: The journey so far and challenges ahead[J]. *IEEE Software*, 2018, 35(3): 24-35.
- [35] KUMAR C. Microservice Architecture: A Paradigm for Accelerating Software Development and Enhancing System Intelligence[J]. *Journal of Computer Science and Technology Studies*, 2025, 7(6): 330-337.
- [36] VELEPUCHA V, FLORES P. A survey on microservices architecture: Principles, patterns and migration challenges.[J]. *IEEE Access*, 2023.
- [37] 张齐勋, 吴一凡, 杨勇, 等. 微服务系统服务依赖发现技术综述[J]. *软件学报*, 2024, 35(01): 118-135. DOI: 10.13328/j.cnki.jos.006827.

-
- [38] VITHARANA P, DAYA S A. Adopting and Sustaining Microservice-based Software Development: Organizational challenges can be more difficult than technical ones.[J]. Communications of the ACM, 2024.
- [39] XIE Z, PEI C, LI W, et al. From Point-wise to Group-wise: A Fast and Accurate Microservice Trace Anomaly Detection Approach[C]//Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. [S.l. : s.n.], 2023: 1739-1749.
- [40] KALDOR J, MACE J, BEJDA M, et al. Canopy: An end-to-end performance tracing and analysis system[C]//Proceedings of the 26th symposium on operating systems principles. [S.l. : s.n.], 2017: 34-50.
- [41] 王璐, 姜宇轩, 李青山, 等. 微服务故障检测研究综述[J]. 计算机学报, 2023, 46(11): 2342-2369.
- [42] NASSIF A B, TALIB M A, NASIR Q, et al. Machine learning for anomaly detection: A systematic review[J]. Ieee Access, 2021, 9: 78658-78700.
- [43] ZHANG S, ZHANG J, LIU G, et al. Bridging Edge and Cloud: A Knowledge-Enhanced Framework for Efficient Time Series Anomaly Detection[J]. IEEE Transactions on Services Computing, 2025.
- [44] LI B, PENG X, XIANG Q, et al. Enjoy your observability: an industrial survey of microservice tracing and analysis[J]. Empirical Software Engineering, 2022, 27: 1-28.
- [45] MA M, XU J, WANG Y, et al. Automap: Diagnose your microservice-based web applications automatically[C]//Proceedings of The Web Conference 2020. [S.l. : s.n.], 2020: 246-258.
- [46] LEE C, YANG T, CHEN Z, et al. Eadro: An end-to-end troubleshooting framework for microservices on multi-source data[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). [S.l. : s.n.], 2023: 1750-1762.
- [47] ZHANG S, JIN P, LIN Z, et al. Robust failure diagnosis of microservice system through multimodal data[J]. IEEE Transactions on Services Computing, 2023, 16(6): 3851-3864.
- [48] LI Y, YU G, CHEN P, et al. Microsketch: Lightweight and adaptive sketch based performance issue detection and localization in microservice systems[C]//International Conference on Service-Oriented Computing. [S.l. : s.n.], 2022: 219-236.
- [49] ZHANG Y, GUAN Z, QIAN H, et al. CloudRCA: A root cause analysis framework for cloud computing platforms[C]//Proceedings of the 30th ACM international conference on information & knowledge management. [S.l. : s.n.], 2021: 4373-4382.
- [50] NOTAROP P, CARDOSO J, GERNDT M. A survey of aiops methods for failure management [J]. ACM Transactions on Intelligent Systems and Technology (TIST), 2021, 12(6): 1-45.

-
- [51] LI Z, ZHAO N, LI M, et al. Actionable and interpretable fault localization for recurring failures in online service systems[C]//Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. [S.l. : s.n.], 2022: 996-1008.
- [52] LI M, LI Z, YIN K, et al. Causal inference-based root cause analysis for online service systems with intervention recognition[C]//Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining. [S.l. : s.n.], 2022: 3230-3240.
- [53] 付楠, 程光, 戴广晔, 等. 面向云原生网络的微服务系统故障诊断方法[J]. 中国科学: 信息科学, 2025, 55(08): 1888-1905.
- [54] SUN Y, LUO Y, WEN X, et al. TrioXpert: An Automated Incident Management Framework for Microservice System[C]//2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE). [S.l. : s.n.], 2025: 3239-3250.
- [55] LIU P, WANG L, RANJAN R, et al. A survey on active deep learning: from model driven to data driven[J]. ACM Computing Surveys (CSUR), 2022, 54(10s): 1-34.
- [56] REN P, XIAO Y, CHANG X, et al. A survey of deep active learning[J]. ACM computing surveys (CSUR), 2021, 54(9): 1-40.
- [57] VATTER J, MAYER R, JACOBSEN H A. The evolution of distributed systems for graph neural networks and their origin in graph processing and deep learning: A survey[J]. ACM Computing Surveys, 2023, 56(1): 1-37.
- [58] KHEMANI B, PATIL S, KOTECHA K, et al. A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions[J]. Journal of Big Data, 2024, 11(1): 18.
- [59] KIPF T N, WELING M. Semi-Supervised Classification with Graph Convolutional Networks[C]//International Conference on Learning Representations. [S.l. : s.n.], 2017.
- [60] VELIČKOVIĆ P, CUCURULL G, CASANOVA A, et al. Graph Attention Networks[C]//International Conference on Learning Representations. [S.l. : s.n.], 2018.
- [61] XU K, HU W, LESKOVEC J, et al. How Powerful are Graph Neural Networks?[C]//International Conference on Learning Representations. [S.l. : s.n.], 2019.
- [62] LI Y, ZEMEL R, BROCKSCHMIDT M, et al. Gated Graph Sequence Neural Networks[C]//Proceedings of ICLR'16. [S.l. : s.n.], 2016.
- [63] OYEWOLE G J, THOPIL G A. Data clustering: application and trends[J]. Artificial intelligence review, 2023, 56(7): 6439-6475.
- [64] ZHANG K, ZHANG C, PENG X, et al. Putracead: Trace anomaly detection with partial labels based on gnn and pu learning[C]//2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE). [S.l. : s.n.], 2022: 239-250.

-
- [65] SCHUSTER M, NAKAJIMA K. Japanese and korean voice search[C]//2012 IEEE international conference on acoustics, speech and signal processing (ICASSP). [S.l. : s.n.], 2012: 5149-5152.
 - [66] WOLF T, DEBUT L, SANH V, et al. Transformers: State-of-the-art natural language processing[C]//Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations. [S.l. : s.n.], 2020: 38-45.
 - [67] LI X, CHEN P, JING L, et al. Swisslog: Robust and unified deep learning based log anomaly detection for diverse faults[C]//2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE). [S.l. : s.n.], 2020: 92-103.
 - [68] VELIČKOVIĆ P, CUCURULL G, CASANOVA A, et al. Graph attention networks[J]. arXiv preprint arXiv:1710.10903, 2017.
 - [69] HE S, FENG B, LI L, et al. STEAM: Observability-Preserving Trace Sampling[C]//Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. [S.l. : s.n.], 2023: 1750-1761.
 - [70] CHECHIK G, SHARMA V, SHALIT U, et al. Large scale online learning of image similarity through ranking.[J]. Journal of Machine Learning Research, 2010, 11(3).
 - [71] ZHOU Y. Scalable Clustering: Large Scale Unsupervised Learning of Gaussian Mixture Models with Outliers[D]. The Florida State University, 2023.
 - [72] HUANG D, WANG C D, WU J S, et al. Ultra-scalable spectral clustering and ensemble clustering[J]. IEEE Transactions on Knowledge and Data Engineering, 2019, 32(6): 1212-1226.
 - [73] Online Boutique[EB/OL]. <https://github.com/GoogleCloudPlatform/microservices-demo>.
 - [74] Chaos Mesh[EB/OL]. 2022. <https://chaos-mesh.org>.
 - [75] SI H, PEI C, CUI H, et al. TimeSeriesBench: An Industrial-Grade Benchmark for Time Series Anomaly Detection Models[J]. arXiv preprint arXiv:2402.10802, 2024.

致谢

成长于节奏舒缓、烟火温和的西南边陲小城的，未曾体会过太高强度的竞争压力，人生旅途一路走来安稳平顺，少有波折坎坷。怀揣着幸运与期许，我远赴天津求学，在南开大学，完整度过了七年本硕青春年华。从初见南开园的懵懂新生，到如今即将完成软件工程专业学业、深耕智能运维方向研究的毕业时刻，七年时光匆匆沉淀，回望来路，满是成长、收获与满心感恩。

感谢我的导师张圣林老师和孙永谦老师。蒙以良师，如灯引路。导师们严谨务实的治学态度、深耕行业前沿的专业视野，为我指明了全部研究方向。在智能运维的研究路上，每一个研究环节都离不开导师耐心细致的点拨与指导。跟随老师深耕各项科研项目的历程里，我稳步打磨了快速文献调研分析、算法架构方案设计、学术成果总结撰写的核心能力，也在一次次项目落地中完成了专业能力的沉淀与成长。

感谢辜文蔚老师和李正丹老师，两位老师在论文修改、方案设计等关键环节，为我提供了专业细致的悉心指正，亦在研究生全程给予我诸多宝贵的关怀、支持与帮助。感谢一路相伴同行、共同深耕科研的各位同门同窗，研途漫漫，有幸并肩，彼此扶持鼓励，在求索之路上一同攻克难题、分享收获，这段温暖同行的时光，始终铭记于心。同时感谢实习期间悉心指引我的字节跳动章阳老师、华为苏亚老师，二位业界导师耐心解惑、倾囊相授，助力我将校内软件工程与智能运维理论知识，与一线工业实践深度融合，令我受益匪浅。

感谢我的家人。你们是我人生最坚实的底色与后盾。远赴千里异乡求学的七年，你们始终无条件支持我的每一个选择，默默承担生活的辛劳，从不强求、从不束缚，只愿我平安顺遂、自在成长。正是家乡故土的温暖与家人毫无保留的托举，让我有底气远离家乡、安心求学，在陌生的北方城市扎根成长，也让我始终心底有归处，行路有方向。

最后感谢一路前行的自己。在南开七年没有选择安逸懈怠，而是主动沉下心夯实软件工程专业基础，一步步踏入智能运维领域，啃读文献、编写代码、调试模型、复盘问题。面对科研瓶颈与技术难题时不曾退缩，在异乡生活里慢慢独立成熟，在日复一日的积累中完成自我成长与蜕变。我感恩平顺的过往，也接纳成长的不足，庆幸自己始终脚踏实地、稳步前行，不负七年南开青春，不负年少初心。

男儿立志出乡关，窗外的风景从连绵的群山变成平原与海岸。蓦然回首，才惊觉求学路早已跨过千里。七年津门求学，我始终铭记南开大学“允公允能，日新月异”的校训精神。毕业在即，我将辞别南开、辞别津门，回到西南故土，以所知所学回馈家乡建设。前路漫漫，唯有笃行不怠。

个人简历、在学期间发表的学术论文及研究成果

个人简历

车泽宇，出生于 2000 年 7 月 1 日。在 2023 年毕业于南开大学软件工程专业并获得学士学位。于 2023 年至今在南开大学就读软件工程研究生。

研究生期间发表论文

- Shenglin Zhang, **Zeyu Che**, Zhongjie Pan, Xiaohui Nie, Yongqian Sun*, Lemeng Pan, Dan Pei. LabelEase: A Semi-Automatic Tool for Efficient and Accurate Trace Labeling in Microservices. 2024 International Symposium on Software Reliability Engineering (ISSRE), Tsukuba, Japan, October 28-31, 2024 (CCF 推荐 B 类国际会议)
- Yongqian Sun, Minghan Liang, Shenglin Zhang*, **Zeyu Che**, Zhiyao Luo, Dongwen Li, Yuzhi Zhang, Dan Pei, Lemeng Pan, and Liping Hou. Efficient Multivariate Time Series Anomaly Detection Through Transfer Learning for Large-Scale Software Systems. ACM Transactions on Software Engineering and Methodology (TOSEM), 2024 (CCF 推荐 A 类国际期刊)
- Ruowei Fu, Yang Zhang, **Zeyu Che**, Xin Wu, Zhenyu Zhong, Zhiqiang Ren, Shenglin Zhang*, Feng Wang, Yongqian Sun, Xiaozhou Liu, Kexin Liu, Yu Zhang. LLM-Powered Multi-Agent Collaboration for Intelligent Industrial On-Call Automation. IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025. (CCF 推荐 A 类国际会议)
- Tianyu Cui, Yang Zhang, Shenglin Zhang*, Xin Wu, Yicheng Sui, Liangyan Peng, Yuhe Ji, Feng Wang, Changchang Liu, **Zeyu Che**, Xiaozhou Liu, Yongqian Sun, and Yu Zhang. “From Chaos to Clarity: Log-based Kernel Panic Root Cause Analysis for Large-Scale Cloud Services,” Frontiers of Computer Science (FCS), 2025. (CCF 推荐 B 类国际期刊)
- Dongwen Li, Shenglin Zhang, Yongqian Sun*, Yang Guo, **Zeyu Che**, Shiqi Chen, Zhenyu Zhong, Minghan Liang, Minyi Shao, Mingjie Li, Shuyang Liu, Yuzhi Zhang, and Dan Pei. An Empirical Analysis of Anomaly Detection Issues for Multivariate Time Series. The 34th IEEE International Symposium on Software Reliability Engineering (ISSRE 2023). Florence, Italy, October 2023 (CCF 推荐 B 类国际会议)